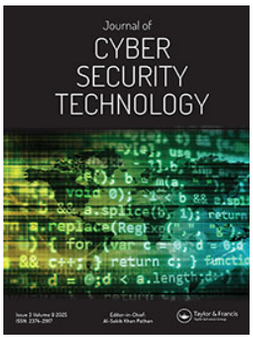

Journal of Cyber Security Technology

ISSN: 2374-2917 (Print) 2374-2925 (Online) Journal homepage: www.tandfonline.com/journals/tsec20

ARM: Adaptive Rank-Based Mutation for Android Malware Detection under Adversarial Attacks

Teenu S. John & Tony Thomas

To cite this article: Teenu S. John & Tony Thomas (18 Aug 2025): ARM: Adaptive Rank-Based Mutation for Android Malware Detection under Adversarial Attacks, Journal of Cyber Security Technology, DOI: 10.1080/23742917.2025.2545641

To link to this article: https://doi.org/10.1080/23742917.2025.2545641

Published online: 18 Aug 2025.

JOURNAL OF CYBER SECURITY TECHNOLOGY
https://doi.org/10.1080/23742917.2025.2545641

ARM: Adaptive Rank-Based Mutation for Android Malware Detection under Adversarial Attacks

Teenu S. John [a] and Tony Thomas[b]

[a]Indian Institute of Information Technology and Management-Kerala, Research Centre of Cochin University of Science and Technology, Cochin, India; [b]Kerala University of Digital Sciences, Innovation and Technology, Thiruvananthapuram, India

ABSTRACT

The escalating threat of Android malware has led to the widespread adoption of machine learning (ML) techniques for detection. While ML-based methods have demonstrated strong performance, they remain vulnerable to adversarial attacks designed to manipulate application features and evade detection. To address this challenge, we propose a robust and adaptive feature selection mechanism based on a Genetic Algorithm enhanced with Rank-Based Adaptive Mutation (GA-RAM). In this method, the mutation rate is dynamically adjusted according to the performance of feature subsets: lower-ranked subsets undergo more aggressive mutations to eliminate features that are likely to be injected by adversaries and are behaviorally irrelevant, while higher-performing subsets are preserved to retain important, semantically meaningful features. This adaptive balance between exploration and exploitation enables the model to converge on resilient and interpretable feature combinations. The proposed mechanism is evaluated under a comprehensive set of adversarial scenarios, including white-box (FGSM, JSMA), grey-box (mimicry, salt-and-pepper noise), black-box (GAN-based), and zero-day attacks. The system achieved 98.6% accuracy in detecting general malware and maintained over 90% accuracy across all adversarial attacks, including 94.1% for zero-day malware. Additionally, SHapley Additive exPlanations (SHAP) are employed to enhance the interpretability and trustworthiness of the detection process.

ARTICLE HISTORY
Received 20 May 2025
Accepted 5 August 2025

KEYWORDS
Android malware detection; adversarial attacks; white-box attacks; black-box attacks; grey-box attacks

1. Introduction

Nowadays, Android smartphone users are threatened by the growing malware attacks [1,2]. In the year 2023, 600 million malicious applications evaded Google Play detection [3]. Most antivirus detection mechanisms employ traditional signature-based detection, in which the application's hash value is compared with the hash values of known malware. When a zero-day attack occurs, the signature-based mechanism cannot detect them [4]. With the revolution of machine learning (ML), cybersecurity practitioners and researchers are utilizing it for advanced cyber threat detection [5,6]. In Android smartphone malware detection, ML based detection mechanisms gave promising results over the past decades. However, these mechanisms can be evaded by an adversary by using various adversarial attacks [7]. The recent study conducted by Kaspersky highlights that although machine learning-based antivirus (AV) systems offer powerful detection capabilities, they are highly susceptible to adversarial manipulation, emphasizing the need for robust defenses [8].

Several studies have explored adversarial attacks on malware detection systems in Android environments [9,10]. Most of the existing adversarial malware are crafted by perturbing the static features. One common tactic is API call injection, where benign-looking API calls are inserted into the code without affecting its functionality. Likewise, attackers employ permission injection by adding benign or commonly requested permissions, which masks the presence of high-risk permissions. Other techniques are API reordering and injecting dead codes to skew statistical detection models to confuse sequence-based models by disrupting the order of significant API calls [11]. More advanced techniques include feature-space adversarial attacks, where malware is crafted using optimization or learning algorithms such as the Fast Gradient Sign Method (FGSM), genetic algorithms, etc [12,13]. to deliberately mislead detectors by exploiting their decision boundaries. Further, malware may also employ obfuscations such as identifier

CONTACT Teenu S. John teenu.john@iiitmk.ac.in Indian Institute of Information Technology and management-Kerala, Research Centre of Cochin University of Science and Technology, Cochin, Thiruvananthapuram 695581, India

renaming, reflection, and string encryption etc. to evade detection. These adversarial attacks make static malware detection mechanisms ineffective, despite their benefits such as high code coverage and the capacity to identify run-time evasive malware. There are several adversarial defense mechanisms proposed in literature. Among them, adversarial training enhances robustness but depends heavily on large, diverse datasets and often overfits to known attack types, making it ineffective against novel adversarial strategies [14]. Although feature squeezing strategies [15] and input sanitization reduce the attack surface, they are likely to remove important behavioral features, leading to degraded accuracy or increased false positives. Graph-based methods on the otherhand capture structural and semantic relationships [16–18] but remain vulnerable to call graph obfuscation and incur high computational costs [19]. Ensemble learning mechanisms [9] improve resilience by combining multiple detectors, but are resource intensive and still susceptible to transferable adversarial examples. Therefore, a mechanism that is computationally less intensive and capable of combating various adversarial attacks has become the need of the hour.

While crafting the attacks, adversaries often prefer feature injections over feature removal, as eliminating core features may disrupt the malware's intended functionality or cause it to crash [20,21]. The injected features are typically behaviorally irrelevant and do not contribute to malicious operations. The existing feature selection mechanisms require manual or heuristic grouping [12] to identify core malicious features which is labour intensive. Ensemble based feature selection on the other hand depends heavily on the choice and tuning of base classifiers, which may not generalize across datasets or malware families [22]. To address these challenges, we propose a Genetic Algorithm enhanced with Rank-Based Adaptive Mutation (GA-RAM) which iteratively evolves feature subsets based on their impact on detection accuracy. In the proposed mechanism, low-ranked chromosomes are subjected to more aggressive mutation, leading to significant alterations in their feature subsets. This increases the chance that irrelevant or features that are likely to be injected will be removed and new, potentially relevant features will be introduced. Feature sets with higher detection accuracy are assigned lower mutation rates, which ensures that effective and semantically meaningful feature combinations are maintained across generations. Over successive iterations, GA with RAM converges on robust and semantically meaningful feature sets by adaptively balancing exploration (through aggressive mutation of low-ranked individuals) and exploitation (by conserving high-performing solutions). The feature set that obtained maximum accuracy are then used to train the classifier for detecting various attacks. In the proposed mechanism, we utilize both API calls and permissions as feature sets for detecting adversarial Android malware. Since permissions requested by an app, declared explicitly in the Android manifest is hard to obfuscate [23], we used them as they can be used to detect obfuscated malware such as those employing string encryption and identifier renaming.

To evaluate the robustness of the proposed detection mechanism, we employed a range of adversarial attack techniques categorized under white-box, grey-box, and black-box threat models. In the white-box setting, we used FGSM and Jacobian-Based Saliency Map Attack (JSMA), which generate adversarial samples by leveraging the model's gradients to perturb features strategically. For grey-box attacks, we applied salt-and-Pepper noise and mimicry attacks. In the black-box setting, we used Generative Adversarial Network (GAN) based attacks, which generate adversarial samples without the knowledge of the model architecture or parameters of the targetted detection model. We also evaluated our mechanism for detecting zero-day malware [24]. Further, we use SHAP [25] to explain the rationale behind the accuracy of our mechanism. The contributions of this work are:

- A novel feature selection mechanism, GA-RAM, is proposed that enhances Android malware detection by adaptively evolving feature subsets through rank-based mutation, effectively identifying and removing features that are likely to be adversarially injected and behaviorally irrelevant while preserving semantically meaningful features.
- The proposed mechanism was comprehensively evaluated across multiple adversarial threat models including white-box, grey-box, black-box, and zero-day attacks and demonstrated high detection accuracy and robustness, with its predictions further supported by SHAP-based interpretability.

The remainder of this paper is structured as follows: Section 2 reviews relevant literature in the field. Section 3 presents a comprehensive overview of adversarial attacks. The proposed methodology is detailed in Section 4 Experimental results are reported in Sections 5 and 6 discusses limitations and future work and Section 7 concludes the paper.

2. Related works

This section details the existing works in Android malware detection and also the adversarial attacks and defense mechanisms implemented till date. The three types of Android malware detection mechanisms are static, dynamic and hybrid. In static mechanisms, the malware is identified with the static features, without executing the application. Dynamic detection mechanisms are those in which the application is made to execute in an isolated environment for extracting its runtime features for examining malicious activities [26]. Hybrid mechanisms on the other hand combine both of these for malware detection. The following sections analyze the existing general and adversarial Android malware attack and detection mechanisms.

2.1. General malware detection mechanisms

Static detection mechanisms take permissions [27,28], API calls [29,30], opcodes [31,32] and other static features for malware detection. Many static malware detection mechanisms rely on computing the similarity between the target application and known benign or malware applications to identify malicious software. However, this approach often leads to high false positive rates, as some legitimate applications may share features commonly associated with malware. There are many deep neural network-based malware detection mechanisms that gave promising results [33–36]. However, the complexity associated with training deep neural network is high. In [27], the mechanism uses permission pairs to detect malware. However, the false positive rate of the mechanism increases significantly due to the occurrence of the same permission pairs appearing in legitimate applications.

To detect malware that executes at runtime, several dynamic mechanisms are also proposed [37,38]. In [39], an ensemble learning mechanism was proposed that used the system level features to detect malware. In [40] battery usage, running processes etc. are used to detect malware. But, their mechanism was not tested on real malware datasets. In [41], the mechanism used layout graphs obtained from user interface traces to detect malicious behaviour. However the mechanism fails if the application has minimal user interface traces. In RepassDroid [42] the mechanism used API call graph of the application to detect malware. But, the effectiveness of the mechanism for detecting adversarial attacks are not evaluated. There are several mechanisms that used frequencies of system calls to detect malware [43]. However, when an app is repackaged it may contain certain system calls that are frequently seen in legitimate application causing misclassification. In [44], a malware detection mechanism with Long Short Term Memory(LSTM) model with system calls as features was proposed. However, the lack of preprocessing may induce noise in the detection mechanism.

There are various hybrid malware detection mechanisms that utilize the advantages of static and dynamic mechanisms [45,46]. In [47], a hybrid detection mechanism was developed. But the effectiveness of the risk assessment mechanism under various attack scenarios was not tested. In HADM [48], a hybrid detection mechanism was proposed with SVM.

There are a number of challenges associated with these malware detection mechanisms. Static detection methods struggle with high false positives [27], while dynamic methods are hindered by reliance on synthetic datasets and incomplete feature extraction, such as user interface traces. Hybrid models, though combining the strengths of both, increase computational overhead and often lack resilience against adversarial attacks.

2.2. Adversarial attacks and detection

This section outlines existing adversarial Android malware attacks and their corresponding defense mechanisms.

2.2.1. Adversarial attacks

In [49], Pierazzi et al. crafted adversarial malware by adding code transplantations to evade detection. Bala et al. [20] proposed a poisoning attack by mutating API calls and permissions to evade detection. In [50], a semiblackbox attack against deep learning-based Android malware detection using simulated annealing algorithm for evasion was proposed. Xu et al. [13] proposed an algorithm using genetic algorithm and used attention mechanism and JSMA to craft adversarial samples. Li et al. [51] proposed an adversarial attack based on a bi-objective Generative Adversarial Network (GAN) to evade both Android malware detection systems and (firewalls).

In conclusion, the rise of adversarial attacks, such as problem space, poisoning, and semiblackbox attacks, has exposed significant vulnerabilities in existing Android malware detection mechanisms. Techniques like modifying code, mutating API calls, or leveraging algorithms such as simulated annealing and GANs have demonstrated the ability to bypass current defenses.

2.2.2. Adversarial detection

There exists several adversarial Android malware detection mechanisms [52,53]. Most of them employ adversarial retraining [54,55], adversarial feature selection [12,56,57] and ensemble learning [58]. In [59], the authors proposed a feature selection mechanism with deep reinforcement learning and recurrent neural networks. However, implementing these methods for feature selection requires significant computational resources. Moreover, interpreting the reasoning behind deep reinforcement-based feature selections is difficult. In [60] the authors proposed DANdroid, an adversarial learning mechanism. Their mechanism was only tested for obfuscated attack detection. In [61] the mechanism used adversarial training. But the generated samples were not tested to validate whether they preserved the malicious nature. In [62], a mechanism called SecCLS was proposed that used a feature selection mechanism. Their mechanism was not tested for detecting mimicry attacks. In [63], the authors proposed an ensemble learning mechanism to detect adversarial attacks. But their mechanism was evaluated only for ransomware. In [64], a mechanism was proposed to detect adversarial malware with GAN. However, their mechanism was not tested with diverse adversarial samples. In [65], the mechanism proposes adversarial training to detect adversarial samples. However, training causes high overhead and can only detect a few adversarial attack types. In [14], the mechanism proposes q-learning to detect adversarial samples. However, their mechanism only considered permission injection. In [66], various ML models are evaluated to detect zero day attacks in Android. In [67], the authors combined zero shot learning with deep learning to detect Android malware, but yielded low performance.

2.3. Zero day Android malware detection

In [68], a zero day malware detection mechanism was proposed by using control flow graphs. However, feature injections can alter the control flow of the application. In [69], the authors proposed a multiview deep learning mechanism. However, their mechanism was evaluated on older datasets, and failed to evaluate the accuracy of the model on latest Android malware samples. In [70], the mechanism proposed a mechanism that used static features to detect zero day malware. However, identifying critical malware features among many features remains a challenge. In [71], the authors proposed that, deep learning models for zero day malware detection suffer from lack of interpretability reducing the trust of the prediction.

3. Adversarial attacks in Android malware detection

Adversarial attacks in machine learning (ML) are generally classified into white-box, black-box, and grey-box categories, based on the level of knowledge an attacker has about the target model [72]. In a white-box attack, the adversary possesses complete access to the model's architecture, parameters, and gradients [73–75]. This allows for precise and targeted perturbations of input features, typically using gradient-based methods such as FGSM or the JSMA, which craft adversarial inputs by maximizing the model's loss while keeping changes minimal and imperceptible.

In contrast, black-box attacks occur when the adversary has no information about the internal details of the model, such as its architecture or gradients [76]. Instead, the attacker probes the model by submitting inputs and analyzing the resulting outputs to infer decision boundaries and manipulate features accordingly [77]. GAN-based attacks are such types of attacks [78].

Grey-box attacks represent an intermediate threat model, where the attacker has a few information of the system, such as limited access to training data, the feature space, or the model's outputs. Two prominent examples of grey-box, gradient-free attacks are salt-and-pepper noise and mimicry attacks [9]. In salt-and-pepper attack, attackers perturb malware feature by adding noise in the form of irrelevant feature. Mimicry attacks [9], on the other hand, modify malware samples so that their feature representations closely resemble those of benign applications. Since both methods rely on knowledge of the

input feature space and access to output feedback but not to internal parameters, they are characterized as grey-box attacks.

In this paper, we create adversarial malware using FGSM, JSMA, GAN, to evaluate the resilience of our proposed detection mechanism under diverse threat models. We also test the detection performance of the proposed mechanism in the presence of salt-and-pepper and mimicry attack samples of [9] and zero-day Android malware.

3.1. Mathematical formulation of adversarial attacks

Consider an application, which can either be benign or malicious. Let $x = (x_1, x_2, \ldots, x_n)$ denote its original feature vector, where $x_i \in \{0, 1\}$ represents the presence ($x_i = 1$) or absence ($x_i = 0$) of the i^{th} feature. Let $y \in \{0, 1\}$ be the label of the application, where $y = 0$ denotes a benign sample, and $y = 1$ denotes a malicious sample. The aim of the adversary is to evade detection by various feature injections, by preserving core malicious features. To simulate the attacks, we flip features from 0 to 1 rather than from 1 to 0. The adversarial attacks simulated in our proposed mechanism are crafted as follows.

3.1.1. FGSM

In this paper, we use binary, additive-only variant of the Fast Gradient Sign Method (FGSM). This is computed as,

$$x^{\text{adv}} = \max(x, \mathbb{I}\,[x + \varepsilon \cdot \text{sign}(\nabla_x J(x, y)) \geq 0.5]) \tag{1}$$

where x^{adv} denotes the adversarial feature vector generated from the original feature vector $x = (x_1, \ldots, x_n)$, ε is the perturbation magnitude, $_xJ(X, Y)$ is the gradient of the loss with respect to the input features.

3.1.2. JSMA

In this paper, we adopt the binary, variant of the Jacobian-based Saliency Map Attack (JSMA), where only inactive features ($x_i = 0$) are modified. The attack computes a saliency map that identifies the most influential input features for misclassification. The saliency map is defined as

$$S(x, 0)[i] = \begin{cases} 0, & \text{if } \frac{\partial F_0(x)}{\partial x_i} < 0 \text{ or } \sum_{j \neq 0} \frac{\partial F_1(x)}{\partial x_i} > 0 \\ \frac{\partial F_0(x)}{\partial x_i} \cdot \left| \sum_{j \neq 0} \frac{\partial F_1(x)}{\partial x_i} \right|, & \text{otherwise} \end{cases} \tag{2}$$

where $F_0(x)$ and $F_1(x)$ denote the classifier's output probabilities for classes $y = 0$ and $y = 1$, respectively. The adversarial feature vector x^{adv} is then generated by modifying the most salient inactive feature:

$$x_i^{\text{adv}} = \begin{cases} 1, & \text{if } x_i = 0 \text{ and } i = \arg\max_j S(x, 0)[j] \\ x_i, & \text{otherwise} \end{cases} \tag{3}$$

This process is repeated iteratively, modifying one feature at a time, until misclassification is achieved or a predefined perturbation limit is reached.

3.1.3. Mimicry and salt-and-pepper

Mimicry attacks aim to modify a malware sample such that its feature distribution closely resembles that of a benign application. This is achieved by selectively activating features present in benign samples. On the other hand, salt-and-pepper attacks introduce noise into the feature space by randomly adding benign or irrelevant features (e.g. via dead code insertion) to a malicious application. Both strategies attempt to evade detection by shifting the malicious sample toward the benign region in the feature space. The adversarial feature vector $x^{\text{adv}} = (x_1^{\text{adv}}, x_2^{\text{adv}}, \ldots, x_n^{\text{adv}})$ is generated by perturbing the original feature vector $x = (x_1, x_2, \ldots, x_n)$ as follows:

$$x_i^{\text{adv}} = \begin{cases} x_i, & \text{if } x_i = 1 \\ 0 \text{ or } 1, & \text{if } x_i = 0 \end{cases} \tag{4}$$

subject to the constraint $|x^{\text{adv}}| > |x|$, where $|x| = \sum_{i=1}^{n} x_i$ denotes the Hamming weight of the feature vector. This increase in Hamming weight reflects the additive nature of the perturbation, where inactive features ($x_i = 0$) are flipped to active ($x_i^{\text{adv}} = 1$) to fool the classifier.

3.1.4. GAN-based attacks

In GAN-based attacks, a Generative Adversarial Network (GAN) is used to create adversarial malware. The GAN consists of a generator G and a discriminator D, which are trained in a minimax game. The generator learns to produce adversarial feature vectors resembling real malware, while the discriminator learns to distinguish real malware from generated ones. The training objective of the GAN is defined as

$$\min_G \max_D V(D,G) = \mathbb{E}_{x\sim p_{\text{data}}}[\log D(x+\eta)] + \mathbb{E}_{z\sim p_z}[\log(1 - D(G(z)+\eta'))] \tag{5}$$

where $x\sim p_{\text{data}}$ is a real malware feature vector, and $z\sim p_z$ is a noise vector sampled from a standard normal distribution. The generator $G(z)$ produces adversarial malware samples, while the discriminator $D(x)$ outputs the probability that input x is a real malware. Small Gaussian perturbations $\eta, \eta'\sim\mathcal{N}(0,\sigma^2)$ are added to real and generated samples, respectively, to stabilize training.

The discriminator is optimized to correctly distinguish real from generated samples using the following loss:

$$\mathcal{L}_D = \mathbb{E}_{x\sim p_{\text{data}}}[\text{BCE}(D(x+\eta), 0.9)] + \mathbb{E}_{z\sim\mathcal{N}(0,I)}[\text{BCE}(D(G(z)+\eta'), 0)] \tag{6}$$

Here, the real malware samples are softly labeled with 0.9 to improve generalization, while generated samples are labeled as 0 to represent adversarial inputs.

The generator is trained to evade the discriminator into classifying adversarial malware as real by minimizing the following loss:

$$\mathcal{L}_G = \mathbb{E}_{z\sim\mathcal{N}(0,I)}[\text{BCE}(D(G(z)), 1)] \tag{7}$$

This encourages the generator to synthesize malware feature vectors that resemble real malware and evade the discriminator. The generator aims to produce outputs that are classified as real malware samples with $y = 1$, and this is achieved by minimizing the binary cross-entropy (BCE) loss between $D(G(z))$ and the target value 1. This objective encourages the generator to craft adversarial malware samples that closely resemble real malware in the feature space, making them harder to detect. The adversarial malware sample is finally given by:

$$x^{\text{adv}} = G(z) \tag{8}$$

where x^{adv} denotes the adversarial feature vector generated by the GAN.

All these adversarial attacks provided above result in:

(1) Feature Expansion: The extended feature set includes new features that were not originally present.
(2) Critical Feature Preservation: Despite the perturbations, critical features necessary for malware functionality, such as specific APIs or permissions, remain unchanged.
(3) Classifier Evasion: These perturbations are crafted to shift the classifier's decision boundary, causing the malicious sample to be classified as legitimate.

4. Proposed method

Figure 1 shows the workflow of the proposed method. After extracting the API calls and permissions from the Android applications, the proposed method combines Mutual Information (MI) and the GA-RAM to detect general, adversarial and zero-day malware by addressing their characteristic feature perturbations. MI identifies an initial feature set by evaluating the mutual information between Android application features, such as APIs and permissions, and their corresponding labels (benign or malicious). GA-RAM then refines this set using evolutionary principles to generate an optimized subset that effectively detects adversarial perturbations.

4.1. Mutual information based feature selection

The feature space of Android applications is large [79] and it is essential to compute the relevant features for malware detection. In Android, each API call corresponds to a specific function that the application can evoke such as accessing the internet, reading files to more sensitive functions such as retrieving the user's location, sending SMS messages, accessing the camera etc. Each of these

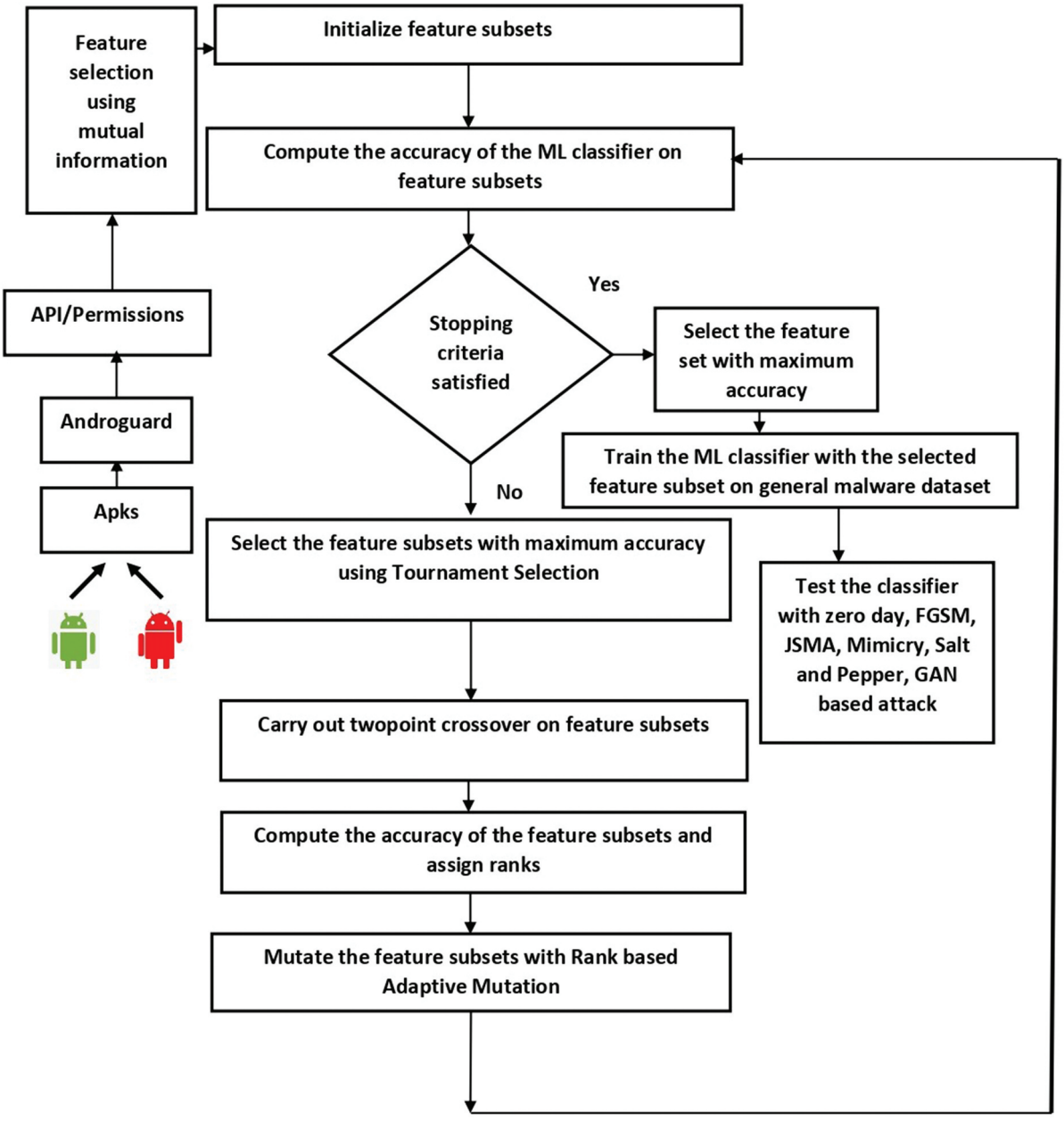

Figure 1. Workflow of the proposed mechanism.

functionalities are protected by specific permissions. As new features are introduced in Android updates, additional permissions are created expanding the permission space making it difficult to detect the most relevant features for malware detection. Hence we use Mutual information (MI) to identify the relevant APIs and permissions that can detect malware. MI filters out the most relevant features by computing their dependency on the target class, ensuring that the chosen features have a strong ability to distinguish between benign and malicious samples. MI has shown excellent results for identifying general malware and adversarial malware in Android applications [80,81]. The mutual information is computed as follows.

Let $X = \{x_1, x_2, \ldots, x_m\}$ denotes the set of features extracted from an Android application, and let $Y = \{y_1, y_2, \ldots, y_m\}$ represents the corresponding labels, where $y_i \in \{0, 1\}$ indicates whether the application is benign ($y_i = 0$) or malicious ($y_i = 1$). The mutual information $I(X; Y)$ measures the dependence between the features X and the labels Y. This metric is utilized to identify the relevant features that are instrumental in identifying between goodware and malware. The mutual information is computed as,

$$I(X;Y) = \sum_{x\in X}\sum_{y\in Y} p(x,y)\log\left(\frac{p(x,y)}{p(x)p(y)}\right) \tag{9}$$

where $p(x,y)$ is the joint probability mass function of the feature x and its label y, $p(x)$ and $p(y)$ are the marginal probability mass functions of x and y respectively. The k-best features are obtained by maximizing the mutual information gain as follows. Let **S** be any non empty subset of $\{x_1, x_2, \ldots, x_m\}$ and Y_S denotes the sets of corresponding labels. The best feature subset **S** is determined as,

$$\arg\max_{\mathbf{S}} I(S;Y_S).$$

When $I(S;Y_S)$ is high, it indicates that the feature subset **S** is highly relevant for distinguishing a malware from a benign application. The k best features obtained using the mutual information are given as the input to the GA-RAM.

4.2. Genetic algorithm with rank based adaptive mutation for feature selection

The relationship between APIs and permissions in Android can be used to identify the behaviour of the applications. For instance, if an app wants to send an SMS, it would use an API call such as SmsManager. sendTextMessage(). However, this API is protected by a permission called SEND_SMS to ensure that only apps that have been granted the specific permission can invoke this. Another example of a potentially malicious API and permission combination is the READ_PHONE_STATE permission and the TelephonyManager.getDeviceId() API. Likewise, malware should contain certain permission pairs for a successful attack [27]. For instance, certain permission pairs such as INTERNET and READ_CONTACTS is used by the spyware to read the user's contact list and send that data to a remote server. During the creation of adversarial malware, these critical permission pairs and API-permission combination often remain unaltered. This is because altering such essential features would likely compromise the core functionality of the malware. The following section explains how GA-RAM helps to identify critical features with its evolutionary operations.

4.2.1. Initializing the feature sets and computing the fitness

In order to detect malware, the initialization step in GA-RAM generates an initial population of feature subsets, each representing a combination of 155 features (APIs and permissions) pre-selected using mutual information. A '1' in these subsets indicates that the corresponding feature is selected, while a '0' means it is not. The algorithm initializes 50 random feature subsets, allowing the GA-RAM to explore diverse combinations of features. By using accuracy as the fitness criterion, the GA-RAM iteratively refines the feature sets over multiple generations. Feature subsets that achieve the highest detection accuracy are selected using Tournament selection [82]. Tournament selection is a selection process commonly used in genetic algorithms where a group (or 'tournament') of feature subsets is first randomly selected, and then the best-performing subset based on its accuracy. The accuracy of the feature subsets are tested with general malware datasets [83,84].

4.2.2. Generating feature subsets that contain malware functionality

After the tournament selection, the crossover operator is applied to the feature subsets to combine features from different subsets to generate new ones. In the context of adversarial malware detection, this process is critical for creating robust feature combinations capable of detecting sophisticated attack strategies. For example, if one feature subset contains API-related features and the other contains key permissions, the crossover operator merges these subsets, resulting in a new feature subset that incorporates both API and permission-related features.

However, existing single-point crossover mechanisms commonly used in GA for Android malware detection [85,86] are less effective in this context. Single-point crossover tends to preserve the sequence of features on one side of the crossover point, limiting the exploration of diverse feature combinations [87]. This lack of diversity can reduce the algorithm's ability to detect adversarial malware, which often requires the identification of interactions between critical API's and permission combinations. Figure 2

shows this. To evaluate the effectiveness of single point and two point crossover in a Genetic Algorithm (GA)-based feature selection framework, we collected 7000 malware applications from Drebin [83,84] and Virusshare [88] and 63,000 benign applications from Google [3]. Each sample was represented as a 155-dimensional binary feature vector, where each feature denotes the presence (1) or absence (0) of a specific static attribute such as API calls or permission requests. We used population size of 50 for 50 generations. The mutation rate was set to 0.1. Both used tournament selections. To evaluate the fitness of each feature subset, we employed classifiers such as Random Forest (RF), Support Vector Machine (SVM), and a Neural Network (NN). Among these, the Random Forest classifier yielded the highest performance. Classification accuracy was used as the fitness score, with an 80:20 train-test split applied across the experiment. From the Figure 2, we can see that two point crossover outperforms the single point crossover mechanism. Our proposed mechanism employs a two-point crossover operator, which significantly enhances the construction of critical feature subsets that adversaries tend to preserve. The 2-point crossover operator enhances the exploration of diverse feature combinations by selecting two points in the parent feature subsets and swapping the features between these points to generate new subsets. This approach allows features from different regions of the parent subsets to combine, resulting in greater diversity in the offspring feature subsets. In the context of adversarial malware detection, this diversity is particularly valuable as it enables the mechanism to capture complex interactions between critical API-related and permission-related features.

4.2.3. Rank based adaptive mutation

After the 2-point crossover, the resulting feature subsets contain highly relevant features that contain relevant permissions and API calls. However, applying uniform mutation to these subsets may cause some '1's to change to '0's, potentially leading to the loss of critical features. To address this challenge, we propose to employ Rank based Adaptive Mutation (RAM) [89]. Unlike uniform mutation, which applies a fixed mutation rate on all the feature subsets, rank-based adaptive mutation adjusts the mutation rate based on the fitness ranking of each feature subsets. In this approach, the ranking is assigned such that the first rank corresponds to the feature subset with the lowest accuracy, which is subjected to the maximum mutation probability p_{max}. The remaining feature subsets are arranged in ascending order of their fitness, with the best-performing subset receiving the last rank. The subset with the highest accuracy is assigned a mutation probability of 0, ensuring that critical features that showed optimal performance are preserved [90]. In the proposed approach, chromosomes with lower fitness are subjected to more aggressive mutation, resulting in substantial changes to their feature compositions. This increases the likelihood of eliminating irrelevant features while introducing new,

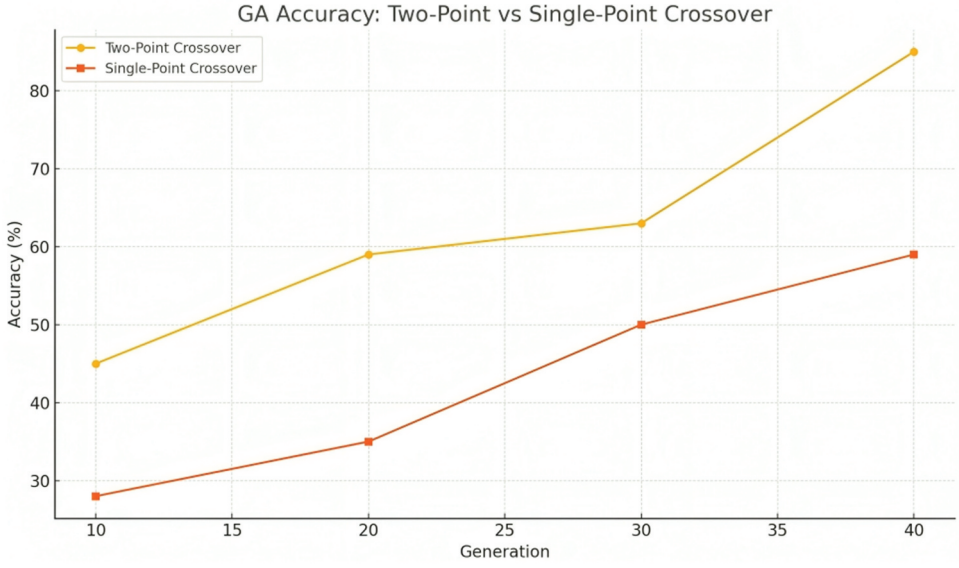

Figure 2. Accuracy values of single point and two point crossover mechanisms.

potentially informative ones. Conversely, feature sets that yield higher detection accuracy undergo minimal mutation, thereby preserving effective and semantically meaningful combinations across generations. Through successive iterations, the GA enhanced with Rank-Based Adaptive Mutation (RAM) progressively converges toward robust feature sets by striking a dynamic balance between exploration by intensified mutation of poorly performing individuals and exploitation by retaining high-performing solutions. Let p_{max} denotes the maximum mutation probability. Suppose there are n feature subsets which are arranged in the increasing order of accuracy (in detecting malware). The mutation probability p_i for the i^{th} feature subset is computed as,

$$p_i = p_{\max} \times \left(1 - \frac{i-1}{n-1}\right) \tag{10}$$

where n denotes the total number of feature subsets. In the next generation, the newly mutated feature subsets are evaluated based on their fitness, which is typically measured by their classification accuracy. The GA-RAM in this way selects the feature subsets with the best performance, prioritizing those that preserve critical permission pairs and API-permission combinations, while discarding or further mutating underperforming subsets. As this process continues across multiple generations, the GA-RAM iteratively improves the population of feature subsets. Algorithm 1 shows the working of GA-RAM. The steps of the algorithm are explained below with reference to its functionality and notation.

Input: $P_s = 50$: Population Size, $N_f = 155$: Number of features, $N_g = 10$: Number of Generations, $c_r = 0.7$: Crossover Probability, $p_{\max} = 0.6$: Mutation Rate, $T = 5$: Stopping Threshold.
Output: Feature set with maximum accuracy $\max(A_j(f_{c''}))$ using 80:20 train-test split on Random Forest classifier
Function GA-RAM($P_s, N_f, N_g, c_r, p_{max}, T$)**:**
  Generate Initial Population $P = (P_s, N_f)$;
  Initialize best accuracy: $A_{\text{best}} \leftarrow 0$;
  Initialize stopping counter: $C_{\text{stop}} \leftarrow 0$;
  for $j = 1$ **to** N_g **do**
    for *each feature set* f_i *in* P **do**
      Evaluate fitness $A_i(f_i)$ using train test split on Random Forest;
    Select feature sets $F_t = \{f_{t_1}, f_{t_2}, \ldots, f_{t_s}\}$ with Tournament Selection;
    Carry out two-point crossover on F_t, $F_c = (F_t, 0.7)$;
    for *each feature set* f_{c_j} *in* F_c **do**
      Evaluate accuracy $A_j(f_{c_j})$ using train test split on Random Forest;
    Sort F_c based on the accuracy: $F_{c'} = \text{Sort}(F_c, A)$;
    for *each feature set* $f_{c'}$ *in* $F_{c'}$ **do**
      Assign rank: $rank(f_{c'})$;
      Compute mutation rate: $p(f_{c'}) = 0.6 \cdot \left(1 - \frac{rank(f_{c'})-1}{P_s-1}\right)$;
      Mutate $f_{c'} \rightarrow f_{c''}$ using $p(f_{c'})$;
    Find the maximum accuracy in current generation: $A_{\text{gen}} = \max(A_j(f_{c''}))$ using train test split and Random Forest;
    if $A_{gen} > A_{best}$ **then**
      Update accuracy: $A_{\text{best}} \leftarrow A_{\text{gen}}$;
      Reset stopping counter: $C_{\text{stop}} \leftarrow 0$;
      Update the population P with mutated feature sets: $P \leftarrow \{f_{c''_1}, f_{c''_2}, \ldots, f_{c''_{P_s}}\}$;
      $j \leftarrow j + 1$;
    else
      Increment stopping counter: $C_{\text{stop}} \leftarrow C_{\text{stop}} + 1$;
    if $C_{stop} < T$ **then**
      Update the population P with mutated feature sets: $P \leftarrow \{f_{c''_1}, f_{c''_2}, \ldots, f_{c''_{P_s}}\}$;
      $j \leftarrow j + 1$;
    else
      Return feature set with maximum accuracy: $\max(A_j(f_{c''}))$ using train test split on Random Forest;
  Return feature set with maximum accuracy $\max(A_j(f_{c''}))$;

Step 1: Initialization

- The algorithm begins by generating an initial population P of size $P_s = 50$. Each feature set f_i within this population is represented as a binary vector containing N_f features where $N_f = 155$.
- The best accuracy observed so far, A_{best}, is initialized to 0. The stopping counter, C_{stop}, is also initialized to 0, with a stopping threshold $T = 5$.
- The genetic algorithm parameters are set with crossover probability $c_r = 0.7$ and maximum mutation rate $p_{\text{max}} = 0.6$.
- Each feature sets accuracy is evaluated using a Random Forest classifier with an 80:20 train-test data split.

Step 2: Fitness Evaluation

- For each feature set $f_i \in P$, the accuracy $A_i(f_i)$ is computed to evaluate its ability to classify benign and malicious applications.
- The accuracy metric measures the quality of f_i in distinguishing between benign ($y = 0$) and malicious ($y = 1$) applications.

Step 3: Selection

- Using tournament selection, a subset of feature sets $F_t = \{f_{t_1}, f_{t_2}, \ldots, f_{t_s}\}$ is selected from the population P.
- This ensures that feature sets with higher accuracy have a greater chance of being selected for crossover.

Step 4: Crossover

- Two-point crossover is applied to the selected feature sets F_t with crossover probability c_r, producing offspring feature sets F_c.
- Crossover combines features from two parent sets to explore new combinations of features.

Step 5: Accuracy Evaluation of Offspring

- For each offspring feature set $f_{c_j} \in F_c$, the accuracy $A_j(f_{c_j})$ is computed.
- This evaluates the fitness of newly generated feature sets for malware detection.

Step 6: Sorting and Ranking

- The offspring feature sets F_c are sorted based on their accuracy, resulting in $F_{c'}$.
- Each feature set $f_{c'} \in F_{c'}$ is assigned a rank, where the most accurate feature set has rank 1.

Step 7: Rank-Based Adaptive Mutation

- The mutation rate $p(f_{c'})$ for each feature set $f_{c'}$ is computed as:

$$p(f_{c'}) = p_{\text{max}} \times \left(1 - \frac{\text{rank}(f_{c'}) - 1}{P_s - 1}\right),$$

where p_{max} is the maximum mutation rate.

- Higher-ranked feature sets undergo smaller mutations, while lower-ranked feature sets are mutated more significantly to explore diverse solutions.
- Each feature set $f_{c'}$ is mutated to produce $f_{c''}$, introducing new variations in the population.

Step 8: Updating the Population

- The maximum accuracy in the current generation, A_{gen}, is identified:

$$A_{\text{gen}} = \max(A_j(f_{c''})).$$

- If $A_{\text{gen}} > A_{\text{best}}$, the best accuracy A_{best} is updated, the stopping counter C_{stop} is reset to 0, and the population P is updated with the mutated feature sets $f_{c''}$.

- Otherwise, the stopping counter C_{stop} is incremented.

Step 9: Stopping Criteria

- The algorithm continues until the stopping counter C_{stop} reaches the threshold T or the maximum number of generations N_g is reached.
- When the algorithm terminates, the feature set with the highest accuracy $\max(A_j(f_{c''}))$ is returned.

4.3. Detection mechanism using ML

After identifying the features with the proposed feature selection, we train a machine learning model (e.g. Random Forest, Support Vector Machine, or Neural Network) using the optimized feature set shown in Figure 1. The model performs classification, identifying the application as either benign or malicious.

5. Experimental results

The experiments were conducted on a Windows 10 machine equipped with an Intel i7 processor. The implementation was carried out using Python.

5.1. Dataset

One thousand five hundred malware applications from the Drebin dataset [84] and 3500 malware applications from the AndroZoo dataset [83], and 2000 malware applications from Virusshare VirusShare were collected. All these malware samples emerged in the year 2012–2019. The malware families used for training are listed in Table 2. Additionally 63,000 benign applications were taken from the Google Playstore [3] The categories of goodware applications used for training are provided in Table 1. We used more benign applications than malware applications to reduce the spatial bias for malware detection as mentioned in [91]. The benign applications were verified using VirusTotal [92] to ensure they were not malicious.

To reduce the temporal bias, for testing, we collected 2500 malware samples from VirusShare [88] and Malwarebazaar [93] that emerged in the year 2020–2022. All the samples were verified with VirusTotal [92] to check whether they exhibited malicious behaviour. The families of malware used for testing are given in Table 3. We also selected 2500 goodware applications from Google Playstore [3] that appeared in the same time frame for testing.

For adversarial attacks, we used the feature peturbation method as mentioned in Section II. We used 1500 malware samples to perturb for FGSM, JSMA and GAN attacks. The malware samples were collected from the Drebin [84] dataset. For salt and pepper attacks, we used adversarial samples of [9]. The salt and pepper attack samples in [9] were generated by adding noise to the malware, including permission injection, dead code injection, etc. These perturbations mimic real-world adversarial modifications to evaluate the robustness of the detection mechanism. For mimicry attacks, the dataset creation followed a 'mimicry × 30' approach, where 30 benign applications were selected. Malware features were injected into each benign application, and the sample requiring the least amount of perturbation was chosen for evaluation [9]. This ensures realistic mimicry attack scenarios. We used an adversarial dataset comprised of 250 mimicry attack samples, 243 salt-and-pepper noise attack samples, along with 1500 benign samples for testing.

Table 1. Benign application categories.

No	Category	Number of Applications
1	Education	7330
2	Entertainment	6500
3	Business	8543
4	Fitness	6345
5	Games	7404
6	Books and Magazines	9005
7	Weather	8108
8	Others	9765

Table 2. Malware families used for training.

Malware	Family	Number of Apps	Repackaged	Obfuscated
Tesbo	SMS Trojan	240	✓	String encryption
FakeInstaller	SMS Trojan	170		String encryption
Mseg	SMS Trojan	200	✓	
DroidKungfu	Backdoor	150		Renaming
Basebridge	Backdoor	110	✓	
Opfake	SMS Trojan	170		
Genimi	Trojan	190		
Golddream	Backdoor	230		String encryption
Rumms	SMS Trojan	130	✓	
Mercor	Trojan Spy	760	✓	Renaming
Boxer	SMS Trojan	370		
MobileTX	Trojan	130	✓	
Plankton	Backdoor	1360	✓	
Vidro	SMS Trojan	960		
Youmi	Adware	300	✓	
Bankbot	Trojan	430		
Adrd	Trojan	270	✓	Renaming
Gingermaster	Backdoor	470		
Jiagu	Adware	320	✓	
Kuguo	Adware	40		Renaming, String encryption

Table 3. Malware families used for testing.

Malware	Family	Number of Apps	Repackaged	Obfuscated
Gumen	SMS Trojan	102	✓	
Airpush	Adware	595		
Lotoor	Exploit	459		Renaming,String encryption
Stealer	Trojan	170		String encryption
Lnk	Trojan	221	✓	Renaming
Andrup	Adware	68		
Minimob	Adware	51	✓	String encryption
Mtk	Trojan	204	✓	
Fjcon	Trojan	68		
NandroBox	Trojan	68	✓	String encryption
Penetho	Exploit	34	✓	Renaming
Boqx	Trojan	17		
AndroRat	Backdoor	51		
Winge	Trojan	85	✓	
Kyview	Adware	68		String encryption
Fakeplayer	SMS Trojan	68		String encryption
Fakedoc	Trojan	68		Renaming,String encryption
Zitmo	Trojan	51		Renaming
Mmarketplay	Trojan	52	✓	Renaming

To evaluate the effectiveness of the proposed detection mechanism in identifying zero-day attacks, we used 1500 latest malware samples from the Virusshare dataset [88] that emerged in the time frame 2023–2024. For testing, we included 1500 benign samples. max width = .5

5.2. Feature extraction and feature selection

Using the Androguard tool, we extracted API calls and permissions from the collected applications. These features were input into the proposed feature selection mechanism, which selected 48 features (as shown in Table 14). The proposed mechanism was evaluated on different machine learning models, and the performance is detailed in Table 4. Among the various ML classifiers, the Random Forest model achieved the highest accuracy. The Random Forest classifier was implemented with sklearn, using 100 decision trees. The fitness was computed using the accuracy obtained after training and testing with the general malware datasets as shown in Tables 2 and 3. Table 5 shows the performance of the proposed mechanism on different parameters such as population size (P_s), number of generations (N_g), crossover probability (c_r) and mutation probability (p_{max}). We achieved the best performance with a population size of 50, a crossover probability of 0.7, a mutation probability of 0.6, for 10 generations. Table 6 shows how the accuracy increases in each generation. We conducted analysis to quantify the retention and

Table 4. Performance of the proposed GA-based feature selection on different ML models.

ML models	Precision	Recall	Accuracy
Random Forest	98.4%	98.8%	98.6%
SVM	96.5%	97.2%	96.8%
Naive Bayes	93.6%	96.5%	95.0%

Table 5. Performance metrics for different GA parameter combinations.

P_s	N_g	c_r	p_{max}	Precision	Recall	Accuracy
20	5	0.7	0.3	85.0	84.5	84.7
30	10	0.8	0.4	89.5	88.8	89.1
50	**10**	**0.7**	**0.6**	**98.4**	**98.8**	**98.6**
70	15	0.9	0.4	94.5	93.8	94.2
100	20	0.8	0.6	92.0	91.5	91.7
150	25	0.7	0.5	95.0	94.7	94.8

Table 6. Fitness of the classifier in each generation.

Number of generations	Accuracy
1	76.5
2	85.2
3	89.5
4	96.7
5	97.1
6	97.5
7	97.9
8	98.0
9	98.2
10	98.6

elimination of 'behaviorally irrelevant' (i.e. likely perturbed) features across generations. These injected features refer to those that were likely to be introduced during adversarial attacks and are not critical to the core malicious behavior. To evaluate this, we tracked the presence of such features across 10 generations of the genetic algorithm. We measured how many of these features were retained versus eliminated as the population evolved under GA-RAM strategy. This is shown in Table 7. The number of retained injected features decreases steadily over generations, indicating that the algorithm effectively eliminates non-discriminative features.

5.3. Performance evaluation

This section presents a comprehensive evaluation of the proposed mechanism's performance in Android malware detection across various scenarios, including general malware, adversarial attacks and zero-day malware.

Table 7. Feature retention vs. elimination over generations.

Generation	Total Features	Retained Features	Eliminated Features
1	155	152	3
2	152	143	9
3	143	131	12
4	131	112	19
5	112	98	14
6	98	85	13
7	85	69	16
8	69	58	11
9	58	51	7
10	51	48	3

5.3.1. Performance comparison on general malware

Table 8 shows the performance of the proposed mechanism on general malware. The proposed mechanism was compared with existing mechanisms, including GA-based approaches [85,97]. The performance comparison in Table 9 highlights the significant superiority of the proposed mechanism over existing approaches for Android malware detection. The proposed method achieves an accuracy of 98.6% on the combination of Drebin, Androzoo and Virusshare dataset, outperforming state-of-the-art methods. Notably, it also demonstrates the lowest false positive rate (FPR) of 2.1% on Drebin, compared to Ficco et al. [94] and Permpair et al. [27], which reported 2.9% and higher. Furthermore, the execution time of 93.4 seconds is markedly lower than that of the existing mechanisms.

To compare GA-RAM with other adaptive mutation mechanisms, we implemented the Gene-Based Adaptive Mutation (GBAM) strategy [99], to benchmark against our proposed GA-RAM framework. Each feature is assigned two separate mutation probabilities p_0 for mutating a 0 to 1 and p_1 for mutating a 1 to 0. These mutation probabilities are dynamically adjusted based on how beneficial a particular feature is to the malware classification performance. Specifically, if the average classification accuracy of feature sets containing a '1'at a given feature exceeds the overall population accuracy (P_{avg}), it implies that the presence of that feature improves malware detection. Consequently, the mutation probability p_0 is increased to favor addition of that feature in future generations, while p_1 is decreased to discourage its removal and vice versa. We used an initial mutation probability $P_{init} = 0.02$, which defines the starting value of both p_0 and p_1 for all features at the beginning of the genetic algorithm. The adaptation step $\gamma = 0.01$ determines the amount by which these probabilities are incremented or decremented during each generation based on the

Table 8. Performance of the proposed mechanism on general and zero day malware.

Mechanism	Type	Dataset	Precision	Recall	Accuracy	FPR	Dataset Size	Training Dataset	Testing Dataset
Proposed Method	General	Drebin+AndroZoo +VirusShare	98.4%	98.8%	98.6%	1.6%	70,000	2012–2019	2020–2022
Proposed Method	Zero-day	VirusShare	97.3%	90.8%	94.1%	2.5%	3,000	2012–2019	2023–2024

Table 9. Comparison of the proposed mechanism with the existing mechanisms.

Mechanism	Type	Dataset	Precision	Recall	Accuracy	FPR	Dataset Size	Features	Model	ExecutionTime (s)
Before Feature Selection	General	Drebin +AndroZoo +VirusShare	86.1%	89.7%	87.6%	14.5%	70,000	Permission+API	Random Forest	123.4
Proposed Method	General	Drebin +AndroZoo +VirusShare	98.4%	98.8%	98.6%	1.6%	70,000	Permission+API	Random Forest	95.9
Proposed Method	General	Virusshare	97.63%	98.8%	98.2%	2.4%	2000	Permission+API	Random Forest	94.7
Proposed Method	General	Drebin	97.9%	98.5%	98.2%	2.1%	1500	Permission+API	Random Forest	90.2
Proposed Method	Zero-day	Virusshare	97.3%	90.8%	94.1%	2.5%	3000	Permission+API	RandomForest	92.5
[94]	General	Drebin	N/A	N/A	93.2%	2.9%	2400	API calls	Ensemble learning	1837
[95]	General	Drebin	N/A	N/A	93.3%	N/A	11,281	Permissions	Deep learning	N/A
[27]	General	Drebin	N/A	N/A	89.1%	4.2%	1500	Permissions	Permission graph	94.8
[96]	General	Virusshare	N/A	N/A	91.2%	N/A	18,000	API graph	Deep learning	893.4
GA based [97]	General	Drebin	94.8	N/A	97.0%	N/A	410	Permissions	GASA-SVM	N/A
GA based [85]	General	Drebin	94.1	N/A	94.1%	N/A	15,036	Permissions	Random Forest	N/A
[69]	Zero-day	Drebin	N/A	N/A	91.0%	N/A	11,120	Opcodes +Permissions +API	Deep learning	N/A
[98]	Zero-day	CICMaldroid2020	83.1%	81.3%	83.0%	N/A	3,68,728	Permissions, system calls etc	Zero shot learning	N/A

Table 10. Comparison of performance metrics between GBAM and GA-RAM.

Metric	GBAM	GA-RAM
Accuracy	90.1%	98.6%
Precision	88.86%	98.4%
Recall	91.7%	98.8%
FPR	11.5%	1.6%
Execution Time (s)	*122.7*	*95.9*

accuracy of the classifier. Feature-level fitness was computed using local accuracy from a Random Forest classifier, evaluated over an 80:20 train-test split with 70,000 samples. The global average fitness P_{avg} was computed using the same model over the entire dataset. However the drawback of GBAM is the need to assign separate mutation probabilities p_0 and p_1 for each gene locus to allow fine-grained adaptation during evolution and as the feature dimensionality increases, this adds to the computational complexity and execution time of the algorithm. Table 10 shows the performance comparison of GBAM with GA-RAM. Compared to GBAM, the proposed GA-RAM framework achieves superior classification accuracy while significantly reducing the computational overhead and execution time, making it more scalable.

To compare the execution time of the proposed mechanism with the other feature selection mechanism, we evaluated a runtime comparison of the GA-RAM with the Support Vector Machine with Recursive Feature Elimination (SVM-RFE) feature selection. The number of features that is to be selected is set as 48. The experiment was conducted on increasing dataset sizes ranging from 70,000 to 100,000 samples. The results demonstrated that GA-RAM consistently outperformed SVM-RFE in terms of execution time across all dataset sizes. For instance, GA-RAM required 95.9 seconds for 70,000 samples compared to 121.2 seconds by SVM-RFE. Similarly, for 100,000 samples, GA-RAM completed in 359.89 seconds, while SVM-RFE took 405.1 seconds. Figure 3 shows this.

5.3.2. Performance comparison on adversarial malware

In this section, we evaluate the effectiveness of the proposed Android malware detection technique against various adversarial attack strategies. Table 11 shows the performance comparison with the existing mechanisms. FGSM was applied under the constraint of additive-only perturbations. The gradient of the cross-entropy loss with respect to the input vector is computed, and each feature is modified in the direction that maximizes the loss as shown in Equation (1). To preserve binary semantics, the perturbed features are thresholded and then merged with the original sample using an element-wise maximum operation. This ensures that no original 1s (malicious features) are removed, maintaining functional consistency while introducing adversarial noise. The value of ε is set to 0.1 and the threshold is set to 0.5.

We employ binary additive-only variant of the Jacobian-based Saliency Map Attack (JSMA) to craft adversarial malware examples. The attack begins with a malware sample represented as a binary feature vector, where each feature corresponds to the presence (1) or absence (0) of an API or permission. First, the gradient of the model's output with respect to each input feature is computed. Based on these gradients, a saliency map $S(X, 0)[i]$ is constructed to identify features whose addition (i.e. flipping from 0 to 1) would most increase the probability of classifying a sample as benign and simultaneously decrease the probability of malware class. Among the features, the one with the highest saliency score is selected and its value is changed from 0 to 1. Equation (2) shows this. This process is iteratively repeated, modifying one feature at a time, until the classifier's prediction changes to benign or a predefined perturbation limit is reached. This method ensures that the core malicious functionality is preserved by only injecting benign-like features and avoiding any removal of key malware characteristics.

For the mimicry and salt-and-pepper attacks, we extracted the API calls and permissions from the adversarial dataset provided in [9], and constructed binary feature vectors where '1' indicates the presence and '0' indicates the absence of a given API call or permission.

For generating adversarial malware samples with GAN, the generator is trained to produce feature vectors that resemble real malware in functionality but are misclassified by the discriminator as benign ($y = 0$). The discriminator is trained using a BCE loss, comparing its predictions on real malware samples (perturbed with Gaussian noise) against a smoothed label value of 0.9 to enhance generalization, and on generated

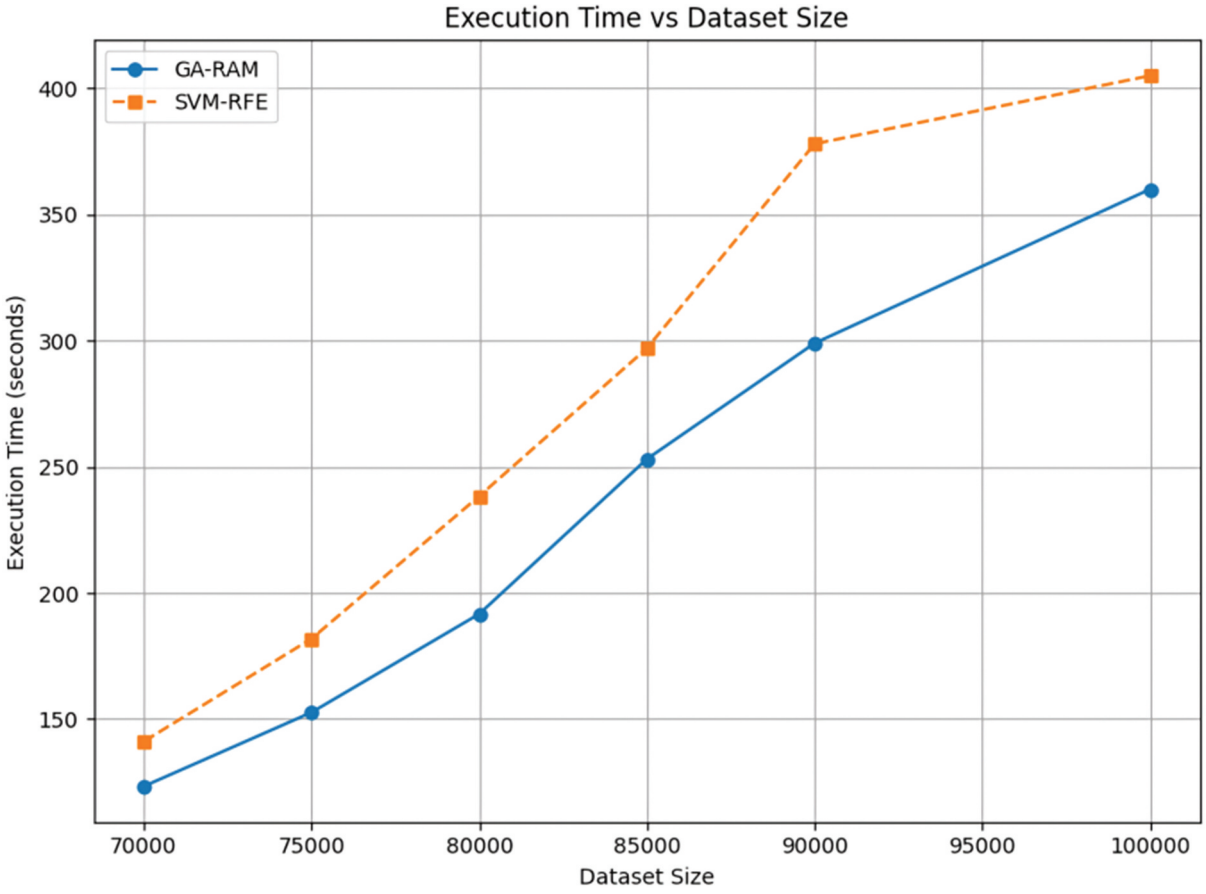

Figure 3. Execution time comparison of GA-RAM with SVM-RFE.

Table 11. Comparision of the proposed mechanism with the existing adversarial detection mechanisms.

Mechanism	Attack	Dataset	Accuracy	FPR	Precision	Recall	FNR	Classifier
Before Feature Selection	FGSM	Drebin	83.0%	12.0%	80.0%	65.0%	35.0%	Random Forest
Before Feature Selection	JSMA	Drebin	69.0%	7.0%	86.5%	45.0%	55.0%	Random Forest
Before Feature Selection	Salt-and-pepper	Drebin	82.1%	9.9%	88.2%	74.2%	25.8%	Random Forest
Before Feature Selection	Mimicry×30	Drebin	75.1%	13.3%	82.6%	63.5%	36.5%	Random Forest
Before Feature Selection	GAN	Drebin	64.7%	7.5%	83.1%	37.0%	63.0%	Random Forest
Proposed Method	FGSM	Drebin	92.3%	7.0%	92.8%	91.7%	8.3%	Random forest
Proposed Method	JSMA	Drebin	93.4%	6.9%	93.1%	93.7%	6.3%	Random forest
Proposed Method	Salt-and-pepper	Drebin	98.4%	2.6%	97.4%	99.5%	0.5%	Random forest
Proposed Method	Mimicry×30	Drebin	96.5%	4.0%	96.0%	97.0%	3.0%	Random Forest
[58]	Salt-and-pepper	Drebin	97.6%	N/A	N/A	N/A	N/A	Deep Learning
[58]	Mimicry×30	Drebin	83.0%	N/A	N/A	N/A	N/A	Deep Learning
[100]	Mimicry×30	Drebin	81.0%	N/A	N/A	N/A	N/A	Deep Learning
Proposed Method	GAN	Drebin	92.9%	7.2%	92.8%	93.1%	6.9%	Random forest

adversarial samples with the true label $y = 1$. As a result, the generator implicitly learns to retain semantically relevant malware features that preserve functionality, while injecting benign-looking noise to evade detection. The generator is optimized to fool the discriminator by minimizing the BCE loss between $D(G(z))$ and the target value $y = 0$, encouraging it to generate malware samples that appear benign in the feature space. This adversarial setup promotes the generation of stealthy malware capable of bypassing detection. The training loss of the GAN is illustrated in Figure 4. A steady decrease in discriminator loss from 1.4172 to 1.2823 indicates that the discriminator is improving in distinguishing real malware samples from adversarial samples. Simultaneously, the increase in generator loss from 0.7146 to 0.8647 reflects that the generator is producing more evasive samples, making the discriminator's task more difficult. The final adversarial dataset, containing these misclassified malware variants, is used to rigorously evaluate the robustness of the

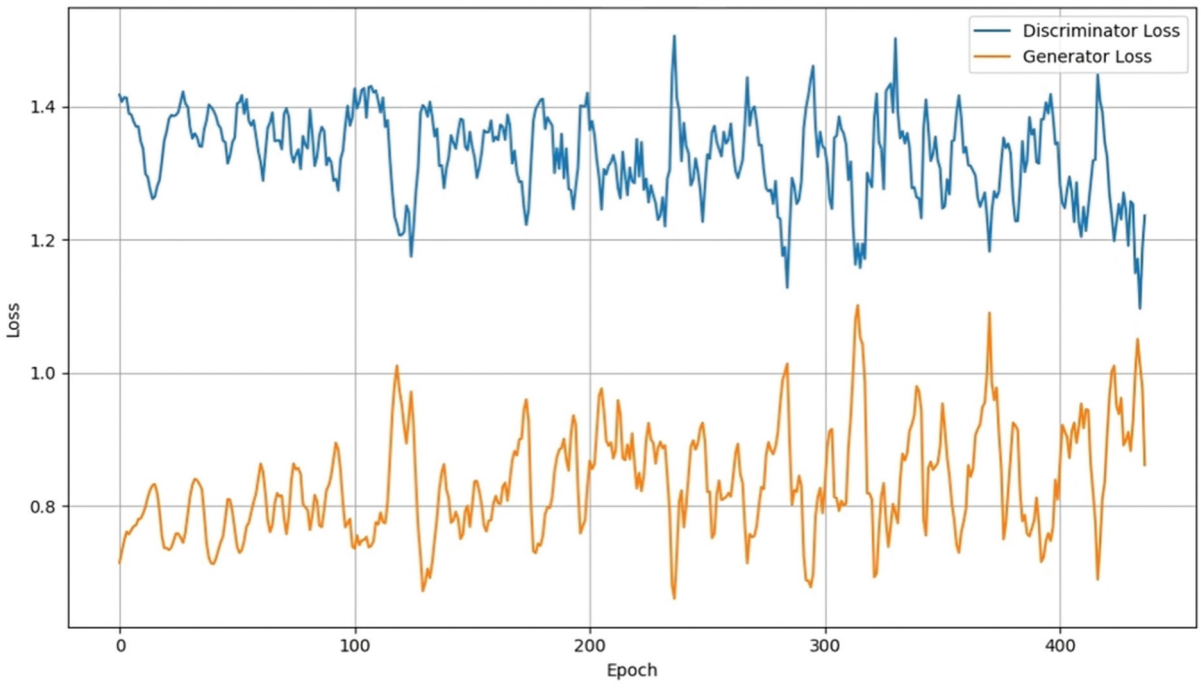

Figure 4. Training loss for GAN.

proposed detection mechanism against GAN-based evasion. The hyperparameters used for GAN training are summarized in Table 12.

To evaluate the effectiveness of the proposed mechanism in comparison with established defense strategies such as adversarial retraining, we conducted experiments using Projected Gradient Descent (PGD) based adversarial retraining, a widely recognized method for enhancing adversarial robustness [101]. In our adversarial retraining setup, we implemented a custom variant of PGD [102] tailored for malware feature vectors under an l_1-norm with $\varepsilon = 5$ as perturbation constraint, step size $\alpha = 0.9$, and 10 iterations per attack, with 3 random restarts to avoid local minima. Perturbations were applied exclusively to malware samples to evaluate the model's robustness under targeted adversarial conditions. To preserve the functional integrity of these malware instances, we enforced feature-preserving constraints that ensured no originally active features were removed during perturbation. Specifically, modifications were restricted to only add new features, thereby preventing any $1 \rightarrow 0$ transitions that could compromise the malicious behavior. All perturbed inputs were ensured to remain within the valid feature range $[0, 1]$. We applied PGD retraining on a general malware classification dataset, as well as across multiple adversarial attack scenarios, including FGSM, JSMA, GAN-based attacks, Mimicry $\times$ 30, and Salt-and-pepper noise. The results of these retraining experiments are summarized in Table 13. Although PGD retraining reduced the evasion to a certain extent, it also resulted in decreased detection sensitivity, particularly in general malware detection scenarios. In contrast, the GA-RAM framework consistently delivered higher recall and superior robustness

Table 12. Parameters for GAN-based adversarial malware generation.

Parameter	Value
Batch Size	64
Generator Learning Rate	0.0002
Discriminator Learning Rate	0.00005
Optimizer	Adam
Generator Activation	ReLU
Discriminator Activation	LeakyReLU
Loss Function	Binary Cross-Entropy
Number of Epochs	400

Table 13. Performance comparison with adversarial retraining.

PGD-retraining	Accuracy	FPR	Precision	Recall	FNR
General malware	85.6%	12.1%	87.3%	83.4%	12.1%
FGSM	87.3%	10.9%	88.7%	85.6%	14.4%
JSMA	89.0%	12.2%	88.0%	90.2%	9.8%
GAN	87.5%	12.0%	87.9%	87.0%	13.0%
Mimicry×30	86.0%	12.0%	87.5%	84.0%	16.0%
Salt-and-pepper	90.0%	9.0%	90.0%	89.9%	10.1%

across a broad range of attack types, underscoring its practical advantages and generalizability in real-world adversarial settings.

5.3.3. Performance on zero-day malware

Table 8 shows the performance of the proposed mechanism on zero day malware. Table 9 shows the comparison of the performance of the proposed detection mechanism in identifying zero-day malware with the existing approaches. For the VirusShareVirusShare dataset, the proposed method achieves an accuracy of 94.1%, a high precision of 97.3%, and a recall of 90.8%, while maintaining a low false positive rate (FPR) of just 2.5%. These results indicate that the method is capable of detecting previously unseen malware samples with high reliability and minimal misclassification of benign apps. In contrast, existing zero-day detection approaches such as Millar et al. [69] and Sawadogo et al. [98] report lower accuracy levels of 91.0% and 83.0%, respectively, and do not provide complete precision or recall statistics. This confirms the robustness and practical applicability of the proposed mechanism for real-world zero-day malware detection scenarios.

5.4. Explanation of the predictions using SHAP

To interpret the classifier's predictions, we employ SHAP values (Shapley Additive Explanations). This section elaborates on the significance of SHAP values, supported by visualizations and comparisons with known malware behavior from the literature.

The plot shown in Figure 5 visualizes the top influential features that is identified by the model to classify a sample as malware. The y-axis lists the features in descending order of their average SHAP value magnitude, while the x-axis represents the SHAP values. Each point on the plot corresponds to a single feature's SHAP value for a sample, with the color scale indicating feature values: red for a value of '1' and blue for a value of '0'. A positive SHAP value indicates that the feature pushes the prediction toward classifying the application as malware, while a negative SHAP value suggests the opposite. For instance, in Figure 5, when the value of loadClass () is '1', its negative SHAP value indicates a decreased likelihood of the sample being classified as malicious. Conversely, features such as READ_PHONE_STATE exhibit positive SHAP values when their value is '1', increasing the likelihood of a malicious classification. Table 14 shows the features that were selected by the proposed mechanism with their corressponding SHAP values.

From Table 14, we can see that the SHAP values derived from our model align with observed malware behavior reported in the literature and public repositories [84,103–106]. Malware families such as Basebridge, Opfake, Gemini, and FakeInstaller often send SMS messages to premium-rate numbers, requiring permissions like SEND_SMS, RECEIVE_SMS, and WRITE_SMS [106]. These features are also used by the latest malware families such as Flubot [107] and Xenomorph [108] to intercept and transmit SMS messages for bypassing multi-factor authentication. Similarly, families like Bankbot, Adrd, DroidKungfu, and Gingermaster collect device information through API calls like getSimSerialNumber() and permissions such as READ_PHONE_STATE [103]. These features also display positive SHAP values, confirming their critical role in classifying malware. Permissions such as RECEIVE_BOOT_COMPLETED, used by families like DroidKungfu, Golddream, Bankbot, and FakeInstaller to activate malicious behavior after device restarts, are also identified by our mechanism with significant SHAP contributions. SpyNote [109], a powerful and recently active Android remote access trojan (RAT), has re-emerged in recent threat campaigns and is considered part of the latest wave of sophisticated Android malware. It exploits permissions such as SYSTEM_ALERT_WINDOW and READ_PHONE_STATE to maintain covert control over infected devices, conduct surveillance, and extract device identifiers for persistent tracking. These same features are identified by the proposed detection mechanism. The API sendTextMessage (SHAP score: 0.139) paired with the permissions SEND_SMS (SHAP

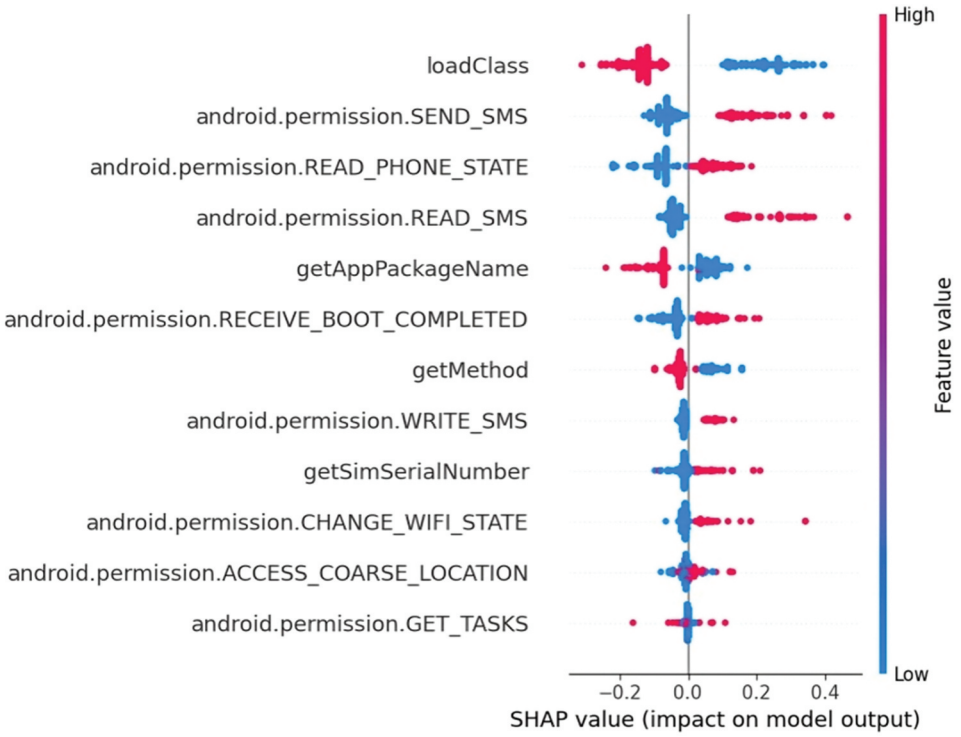

Figure 5. SHAP summary plot.

Table 14. Features selected by adaptive genetic algorithm with SHAP scores.

SI No	Malware Related Features	SHAP Score	SI No	Benign Application Features	SHAP Score
1	SEND_SMS	0.239	1	FACTORY_TEST	−0.220
2	RECEIVE_SMS	0.214	2	loadClass	−0.265
3	READ_PHONE_STATE	0.201	3	requestFocus	−0.213
4	READ_SMS	0.192	4	BROADCAST_STICKY	−0.175
5	RECEIVE_BOOT_COMPLETED	0.184	5	GET_ACCOUNTS	−0.164
6	WRITE_SMS	0.176	6	MODIFY_AUDIO_SETTINGS	−0.152
7	getSimSerialNumber	0.168	7	getAppPackageName	−0.109
8	CHANGE_WIFI	0.163	8	SET_WALLPAPER	−0.161
9	ACCESS_COARSE_LOCATION	0.150	9	EXPAND_STATUS_BAR	−0.158
10	GET_TASKS	0.142	10	WRITE_HISTORY_BOOKMARKS	−0.109
11	sendTextMessage	0.139	11	VIBRATE	−0.105
12	getSimOperator	0.131	12	randomUUID	−0.098
13	getDeviceId	0.128	13	USE_BIOMETRIC	−0.083
14	SYSTEM_ALERT_WINDOW	0.121	14	getMethod	−0.072
15	getSubscriberId	0.115	15	INSTALL_SHORTCUT	−0.064
16	ACCESS_LOCATION_EXTRA_COMMANDS	0.090			
17	READ_CONTACTS	0.109			
18	getMessage	0.102			
19	getInputStream	0.070			
20	getNetworkOperator	0.060			
21	getOutputStream	0.057			
22	CALL_PHONE	0.050			
23	ContentResolver.query	0.046			
24	WRITE_ EXTERNAL_ STORAGE	0.040			
25	KILL_BACKGROUND_PROCESSES	0.030			
26	WRITE_CONTACTS	0.021			
27	DELETE_PACKAGES	0.016			
28	getPackageInfo	0.014			
29	getLine1Number	0.012			
30	ACCESS_FINE_LOCATION	0.010			
31	ACCESS_NETWORK_STATE	0.009			
32	READ_EXTERNAL_STORAGE	0.003			
33	getLastKnownLocation	0.002			

score: 0.239) and WRITE_SMS (SHAP score: 0.176) enables unauthorized SMS communication, which is a common tactic in phishing and premium SMS fraud. Similarly, ContentResolver.query combined with WRITE_CONTACTS and READ_CONTACTS are exploited by spyware for data exfiltration or targeted phishing campaigns. Permissions such as ACCESS_COARSE_LOCATION and ACCESS_FINE_LOCATION, paired with the API getLastKnownLocation, allow malware to track the user. Device related APIs like getSimSerialNumber and getDeviceId, paired with READ_PHONE_STATE are frequently exploited by trojans and adware to extract unique device identifiers for targeted attacks or intrusive advertising practices. File and network operations, such as the combination of getInputStream and getOutputStream with READ_EXTERNAL_STORAGE and WRITE_EXTERNAL_STORAGE, are exploited by the malware to extract data from the device and C&C server communication. Permissions like RECEIVE_BOOT_COMPLETED paired with KILL_BACKGROUND_PROCESSES enable malware to maintain persistence and disrupt system operations. Features such as loadClass (SHAP score: −0.265) and requestFocus (SHAP score: −0.213) show negative SHAP values, indicating their association with benign behaviors. FACTORY_TEST (SHAP score: −0.220) is typically used in device diagnostics or manufacturing scenarios.The GET_ACCOUNTS permission (SHAP score: −0.164) is widely used in cloud-based and synchronization applications to access the list of user accounts on the device. Likewise, MODIFY_AUDIO_SETTINGS (SHAP score: −0.152) is essential for media players, VoIP apps, and system tools that adjust audio profiles. APIs such as getAppPackageName and getMethod, with negative SHAP scores, reflect introspection and logging practices common in benign applications. Features like SET_WALLPAPER, EXPAND_STATUS_BAR,VIBRATE USE_BIOMETRIC, and INSTALL_SHORTCUT are frequently used to enhance the user experience and interface customization.

The SHAP force plot shown in Figure 6 explains the prediction of the proposed model for an instance belonging to a banking trojan. The model predicted this sample as malicious with high confidence ($f(x) = 0.98$), significantly above the base value of 0.46, which represents the model's average prediction across all samples. The strong push toward the malware classification is primarily driven by several highly indicative features. Notably, the sample requests access to sensitive telephony information such as getSimSerialNumber = 1 and android.permission. READ_PHONE_STATE = 1, which are commonly used by banking trojans to uniquely identify a device. Additionally, the permissions android.permission.WRITE_SMS = 1 and android.permission.READ_SMS = 1 suggest the ability to intercept and manipulate SMS messages for credential theft and bypassing two-factor authentication mechanisms. These features together contribute heavily to the model's confidence that the sample is malicious.

Notably, the permissions and API – permission combinations identified by the proposed model are not arbitrary, but represent semantically meaningful operations that are core to the malicious intent of Android malware. Since malware, under adversarial manipulation, must preserve its core functional semantics to remain operational, the proposed detection mechanism is able to identify these persistent malicious features despite such evasive transformations. Hence by correlating SHAP scores with known malware and benign application behaviors and literature, we validate the effectiveness and reliability of our approach.

6. Limitations and future work

The proposed mechanism relies on static features for detection and may be evaded by malware that employs runtime obfuscation and native code execution. Further as Android evolves, certain API's become deprecated or renamed. To overcome these limitations, future work of GA-RAM will incorporate dynamic behavioral features such as system calls [110], memory access patterns, and inter-process communication traces captured during the actual execution of Android applications. By supplementing static features with these run-time features, the mechanism can detect stealthy malware that activates only during execution and those that employs native code execution. Additionally, the integration of opcode-level features, which operate at the Dalvik bytecode level and remain stable across Android API versions, offers resilience against API-specific

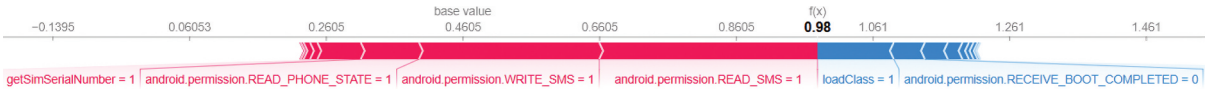

Figure 6. SHAP force plot of banking Trojan.

deprecations and renamings. Hence by combining static and dynamic features, we plan to build a robust malware detection mechanism, capable of identifying evasive and stealthy threats across diverse Android environments.

7. Conclusion

This work presented a robust and adaptive feature selection mechanism, GA-RAM, for enhancing Android malware detection. GA-RAM intelligently balances exploration and exploitation during feature evolution, enabling the identification of semantically meaningful and attack-resilient feature subsets. Extensive evaluations demonstrated the effectiveness of this approach in achieving 98.6% accuracy in detecting general malware, and maintaining over 90% detection accuracy under a variety of adversarial attack settings, including white-box, black-box, grey-box, and zero-day scenarios. The integration of SHAP explanations further enhances the model's transparency by identifying key contributing features, thus promoting interpretability and trustworthiness in predictions.

Disclosure statement

The authors declare that there are no relevant financial or non-financial competing interests to report.

ORCID

Teenu S. John http://orcid.org/0000-0001-5019-8615

Data availability statement

The data that support the findings of this study are available from the corresponding author T.S.John (Teenu S. John), upon reasonable request.

References

[1] Soto-Valero C, González M. Empirical study of malware diversity in major Android markets. J Cyber Secur Technol. 2018;2(2):51–74. doi: 10.1080/23742917.2018.1483876
[2] Krych DE, McDaniel P. Exposing android social applications: linking data leakage to privacy policies. J Cyber Secur Technol. 2021;5(3–4):139–190. doi: 10.1080/23742917.2019.1630093
[3] Google. Android apps on Google Play. 2023. Available from: https://play.google.com/store/games?hl=en_INgl=USpli=1
[4] Ani UD, Watson J, He H, et al. Minimising cybersecurity risk exposures in industrial control system environments: a techno-human vulnerability analysis approach. J Cyber Secur Technol. 2024:1–40. doi: 10.1080/23742917.2024.2421589
[5] Makhdoomi PMS, Ikhlas M, Khursheed A, et al. Network-based intrusion detection: a comparative analysis of machine learning approaches for improved security. J Cyber Secur Technol. 2025:1–28. doi: 10.1080/23742917.2024.2447119
[6] Zhou C. Detection and classification of phishing websites using machine learning approach taking advantage of metaheuristic algorithms efficiency. J Cyber Secur Technol. 2025:1–18. doi: 10.1080/23742917.2025.2492923
[7] John TS, Thomas T. Handbook of Research on Cloud Computing and Big Data Application in IoT. IGI Global; 2019. p. 127–150. doi: 10.4018/978-1-5225-8407-0.ch007
[8] Kaspersky. AI under attack-white paper. [cited 2025 May 5]. Available from: https://content.kaspersky-labs.com/se/media/en/business-security/enterprise/machine-learning-cybersecurity-whitepaper.pdf
[9] Deqiang L, Qianmu L. Adversarial deep ensemble: evasion attacks and defenses for malware detection. IEEE Trans Inf Forensic Secur. 2020;15:3886–3900. doi: 10.1109/TIFS.2020.3003571
[10] Shahpasand M, Hamey L, Vatsalan D, et al. Adversarial attacks on mobile malware detection. In: AI4Mobile 2019–2019 IEEE 1st International Workshop on Artificial Intelligence for Mobile; Hangzhou, China; 2019. p. 17–20. doi: 10.1109/AI4MOBILE.2019.8672711
[11] Chen X, Li C, Wang D, et al. Android HIV: a study of repackaging malware for evading machine-learning detection. IEEE Trans Inf Forensic Secur. 2019;15:987–1001. doi: 10.1109/TIFS.2019.2932228
[12] Li X, Kong K, Xu S, et al. Feature selection-based Android malware adversarial sample generation and detection method. IET Inf Secur. 2021;15(6):401–416. doi: 10.1049/ise2.12030

[13] Xu G, Shao H, Cui J, et al. Gendroid: a query-efficient black-box android adversarial attack framework. Comput Secur. 2023;132:103359. doi: 10.1016/j.cose.2023.103359
[14] Rathore H, Sahay SK, Nikam P, et al. Robust android malware detection system against adversarial attacks using Q-learning. Inf Syst Front. 2021;23(4):867–882. doi: 10.1007/s10796-020-10083-8
[15] Xu W, Evans D, Qi Y. Feature squeezing: detecting adversarial examples in deep neural networks. 25th Annual Network and Distributed System Security Symposium; San Diego, California. 2018. arXiv preprint arXiv:1704.01155.
[16] Li Z, Liu H, Wu C. Computer network security evaluation method based on improved attack graph. J Cyber Secur Technol. 2022;6(4):201–215. doi: 10.1080/23742917.2022.2120293
[17] Liu Z, Wang R, Japkowicz N, et al. Segdroid: an Android malware detection method based on sensitive function call graph learning. Expert Syst Appl. 2024;235:121125. doi: 10.1016/j.eswa.2023.121125
[18] Shen L, Fang M, Xu J. Ghgdroid: global heterogeneous graph-based android malware detection. Comput Secur. 2024;141:103846. doi: 10.1016/j.cose.2024.103846
[19] Zhao K, Zhou H, Zhu Y, et al. Structural attack against graph based Android malware detection. In: Proceedings of the 2021 ACM SIGSAC conference on computer and communications security; Seoul, Republic of Korea; 2021. p. 3218–3235.
[20] Bala N, Ahmar A, Li W, et al. Droidenemy: battling adversarial example attacks for Android malware detection. Digit Commun Networks. 2022;8(6):1040–1047. doi: 10.1016/j.dcan.2021.11.001
[21] Renjith G, Vinod P, Aji S. Evading machine-learning-based Android malware detector for IoT devices. IEEE Syst J. 2022;17(2):2745–2755. doi: 10.1109/JSYST.2022.3215014
[22] Islam R, Sayed MI, Saha S, et al. Android malware classification using optimum feature selection and ensemble machine learning. Internet Things cyber-Phys Syst. 2023;3:100–111. doi: 10.1016/j.iotcps.2023.03.001
[23] Wang H, Zhang W, He H. You are what the permissions told me! Android malware detection based on hybrid tactics. J Inf Secur Appl. 2022;66:103159. doi: 10.1016/j.jisa.2022.103159
[24] Crowdstrike. Zero day. [cited 2024 Oct 11]. Available from: https://www.crowdstrike.com/en-us/cybersecurity-101/cyberattacks/zero-day-exploit/
[25] Štrumbelj E, Kononenko I. Explaining prediction models and individual predictions with feature contributions. Knowl Inf Syst. 2014;41(3):647–665. doi: 10.1007/S10115-013-0679-X
[26] John TS, Thomas T, Emmanuel S. Detection of evasive android malware using Eigengcn. J Inf Secur Appl. 2024;86:103880. doi: 10.1016/j.jisa.2024.103880
[27] Arora A, Peddoju SK, Conti M. Permpair: android malware detection using permission pairs. IEEE Trans Inf Forensics Secur. 2020;15:1968–1982. doi: 10.1109/TIFS.2019.2950134
[28] Felt AP, Chin E, Hanna S, et al. Android permissions demystified. In: Proceedings of the ACM Conference on Computer and Communications Security; Chicago, Illinois, USA; 2011. p. 627–636. doi: 10.1145/2046707.2046779
[29] Aafer Y, Du W, Yin H. DroidAPIMiner: mining API-level features for robust malware detection in Android. In: Tanveer A, Zia A, Zomaya Y, editors. Lecture notes of the Institute for Computer Sciences, Social-Informatics and Telecommunications Engineering. Sydney, Australia: LNICST, Springer Verlag; 2013. p. 86–103. doi: 10.1007/978-3-319-04283-1_6
[30] Yang H, Wang Y, Zhang L, et al. A novel android malware detection method with API semantics extraction. Comput Secur. 2024;137:103651. doi: 10.1016/j.cose.2023.103651
[31] Amin M, Tanveer TA, Tehseen M, et al. Static malware detection and attribution in Android byte-code through an end-to-end deep system. Future Gener Comput Syst. 2020;102:112–126. doi: 10.1016/j.future.2019.07.070
[32] Surendran R, Thomas T, Uddin MM. Optimizing malware detection with redundant sample filtering for efficient retraining. J Cyber Secur Technol. 2024:1–22. doi: 10.1080/23742917.2024.2438696
[33] Elayan ON, Mustafa AM. Android malware detection using deep learning. Procedia Comput Sci. 2021;184:847–852. doi: 10.1016/J.PROCS.2021.03.106
[34] Naeem H, Cheng X, Ullah F, et al. A deep convolutional neural network stacked ensemble for malware threat classification in Internet of Things. J Circuits Syst Comput. 2022;31(17):2250302. doi: 10.1142/S0218126622503029
[35] Naeem H, Dong S, Falana OJ, et al. Development of a deep stacked ensemble with process based volatile memory forensics for platform independent malware detection and classification. Expert Syst Appl. 2023;223:119952. doi: 10.1016/j.eswa.2023.119952
[36] Shu L, Dong S, Su H, et al. Android malware detection methods based on convolutional neural network: a survey. IEEE Trans Emerging Top Comput Intel. 2023;7(5):1330–1350. doi: 10.1109/TETCI.2023.3281833
[37] John TS, Thomas T, Emmanuel S. Graph convolutional networks for Android malware detection with system call graphs. Piscataway, New Jersey: IEEE; 2020. doi: 10.1109/ISEA-ISAP49340.2020.235015
[38] Smmarwar SK, Gupta GP, Kumar S. Android malware detection and identification frameworks by leveraging the machine and deep learning techniques: a comprehensive review. Telematics Inf Rep. 2024;14:100130. doi: 10.1016/j.teler.2024.100130
[39] Feng P, Ma J, Sun C, et al. A novel dynamic Android malware detection system with ensemble learning. IEEE Access. 2018;6:30996–31011. doi: 10.1109/ACCESS.2018.2844349
[40] Shabtai A, Kanonov U, Elovici Y, et al. "Andromaly": a behavioral malware detection framework for android devices. J Intell Inf Syst. 2012;38(1):161–190. doi: 10.1007/S10844-010-0148-X/TABLES/8

[41] Yue S, Feng W, Ma J, et al. RepDroid: an automated tool for Android application repackaging detection. In: IEEE International Conference on Program Comprehension; Buenos Aires, Argentina; 2017. p. 132–142. doi: 10.1109/ICPC.2017.16
[42] Xie N, Zeng F, Qin X, et al. RepassDroid: automatic detection of Android malware based on essential permissions and semantic features of sensitive APIs. In: Proceedings - 2018 12th International Symposium on Theoretical Aspects of Software Engineering; Guangzhou, China. 2018-January, 2018. p. 52–59. doi: 10.1109/TASE.2018.00015
[43] Dimjaševi M, Atzeni S, Rakamari Z, et al. Evaluation of Android malware detection based on system calls. In: IWSPA 2016 - Proceedings of the 2016 ACM International Workshop on Security and Privacy Analytics, co-located with CODASPY 2016; United States; 2016. p. 1–8. doi: 10.1145/2875475.2875487
[44] Xiao X, Zhang S, Mercaldo F, et al. Android malware detection based on system call sequences and LSTM. Multimedia Tools Appl. 2019;78(4):3979–3999. doi: 10.1007/S11042-017-5104-0/FIGURES/9
[45] Dong S, Shu L, Nie S. Android malware detection method based on CNN and DNN bybrid mechanism. Piscataway, NJ, USA: IEEE Transactions on Industrial Informatics; 2024.
[46] Yunmar RA, Kusumawardani SS, Mohsen F, et al. Hybrid android malware detection: a review of heuristic-based approach. IEEE Access. 2024;12:41255–41286. doi: 10.1109/ACCESS.2024.3377658
[47] Song L, Tang Z, Li Z, et al. AppIS: protect Android apps against runtime repackaging attacks/ International Conference on Parallel and Distributed Systems (ICPADS); Shenzhen, China. IEEE; 2017. p. 1521–9097. doi: 10.1109/ICPADS.2017.00015
[48] Xu L, Zhang D, Jayasena N, et al. HADM: hybrid analysis for detection of malware. Lect Notes Networks Syst. 2018;16:702–724. doi: 10.1007/978-3-319-56991-8_51/COVER
[49] Pierazzi F, Pendlebury F, Cortellazzi J, et al. Intriguing properties of adversarial ML attacks in the problem space. In: 2020 IEEE symposium on security and privacy (SP); San Francisco, California. IEEE; 2020. p. 1332–1349.
[50] Xu G, Xin G, Jiao L, et al. Ofei: a semi-black-box android adversarial sample attack framework against DLAAS. IEEE Trans Comput. 2023;73(4):956–969. doi: 10.1109/TC.2023.3236872
[51] Li H, Zhou S, Yuan W, et al. Adversarial-example attacks toward Android malware detection system. IEEE Syst J. 2019;14(1):653–656. doi: 10.1109/JSYST.2019.2906120
[52] Bhusal D, Rastogi N. Adversarial patterns: building robust Android malware classifiers. ACM Comput Surv. 2025;57(8):1–34. doi: 10.1145/3717607
[53] John TS, Thomas T. Evading static and dynamic Android malware detection mechanisms. In: Security in Computing and Communications: 8th International Symposium, SSCC 2020; Chennai, India; 2021.
[54] Valeti K, Rathore H. Gbkpa and auxshield: addressing adversarial robustness and transferability in Android malware detection. Forensic Sci Int: Digit Investigation. 2024;50:301816. doi: 10.1016/j.fsidi.2024.301816
[55] Wang H, Chen T, Gui S, et al. Once-for-all adversarial training: in-situ tradeoff between robustness and accuracy for free. Adv Neural Inf Process Syst. 2020;33:7449–7461.
[56] Li X, Kong K, Xu S, et al. Feature selection-based Android malware adversarial sample generation and detection method. IET Inf Secur. 2021;15(6):401–416. doi: 10.1049/ISE2.12030
[57] Yang P, Chen J, Hsieh CJ, et al. Ml-loo: detecting adversarial examples with feature attribution. In: Proceedings of the AAAI Conference on Artificial Intelligence; New York, USA; 2020. p. 6639–6647.
[58] Li H, Zhou S, Yuan W, et al. Robust android malware detection against adversarial example attacks. In: The Web Conference 2021 - Proceedings of the World Wide Web Conference, WWW 2021; Ljubljana, Slovenia; 2021. Vol. 10. p. 3603–3612. doi: 10.1145/3442381.3450044
[59] Wu Y, Li M, Zeng Q, et al. DroidRL: feature selection for Android malware detection with reinforcement learning. Comput Secur. 2023;128:103126. doi: 10.1016/J.COSE.2023.103126
[60] Millar S, McLaughlin N, Del Rincon JM, et al. Dandroid: a multi-view discriminative adversarial network for obfuscated Android malware detection. In: CODASPY 2020 - Proceedings of the 10th ACM Conference on Data and Application Security and Privacy; Louisiana, USA; 2020. Vol. 12. p. 353–364. doi: 10.1145/3374664.3375746
[61] Wang C, Zhang L, Zhao K, et al. AdvAndMal: adversarial training for Android malware detection and family classification. Symmetry. 2021;13(6):1081. doi: 10.3390/SYM13061081
[62] Chen L, Hou S, Ye Y. SecureDroid: enhancing security of machine learning-based detection against adversarial Android malware. Attacks. 2017. doi: 10.1145/3134600.3134636
[63] Ahmed U, Lin JCW, Srivastava G. Mitigating adversarial evasion attacks of ransomware using ensemble learning. Comput Electr Eng. 2022;100:107903. doi: 10.1016/J.COMPELECENG.2022.107903
[64] Amin M, Shah B, Sharif A, et al. Android malware detection through generative adversarial networks. Trans Emerging Telecommun Technol. 2022;33(2):e3675. doi: 10.1002/ett.3675
[65] Wang C, Zhang L, Zhao K, et al. Advandmal: adversarial training for Android malware detection and family classification. Symmetry. 2021;13(6):1081. doi: 10.3390/sym13061081
[66] Alhaidari F, Shaib NA, Alsafi M, et al. Zevigilante: detecting zero-day malware using machine learning and sandboxing analysis techniques. Comput Intell Neurosci. 2022;2022:1615528. doi: 10.1155/2022/1615528
[67] Palarimath S, Maran P, Kavitha S, et al. Uncovering Android zero-day threats: the zero-vuln approach with deep and zero-shot learning. In: 2024 2nd International Conference on Computing and Data Analytics (ICCDA); Shinas, Oman. IEEE; 2024. p. 1–6.

[68] Grace M, Zhou Y, Zhang Q, et al. RiskRanker: scalable and accurate zero-day Android malware detection. In: Proceedings of the 10th international conference on Mobile systems, applications, and services; United Kingdom; 2012. p. 281–294.
[69] Millar S, McLaughlin N, Del Rincon JM, et al. Multi-view deep learning for zero-day Android malware detection. J Inf Secur Appl. 2021;58:102718. doi: 10.1016/j.jisa.2020.102718
[70] Sara JJ, Hossain S. Static analysis based malware detection for zero-day attacks in Android applications. In: 2023 International Conference on Information and Communication Technology for Sustainable Development (ICICT4SD); Dhaka, Bangladesh. IEEE; 2023. p. 169–173.
[71] Deldar F, Abadi M. Deep learning for zero-day malware detection and classification: a survey. ACM Comput Surv. 2023;56(2):1–37. doi: 10.1145/3605775
[72] Chakraborty A, Alam M, Dey V, et al. Adversarial attacks and defences: a survey. CAAI Transactions on Intelligence Technology. 2018;6(1):25–45.
[73] Goodfellow IJ, Shlens J, Szegedy C. Explaining and harnessing adversarial examples. 3rd International Conference on Learning Representations, ICLR; 2015; San Diego, CA, USA. arXiv preprint arXiv. 2015;1412:6572.
[74] Madry AMakelov A, Schmidt L, et al. Towards deep learning models resistant to adversarial attacks International Conference on Learning Representations; Canada. 2018. arXiv preprint arXiv:1706.06083
[75] Mahindru A, Arora H, Kumar A, et al. Permdroid a framework developed using proposed feature selection approach and machine learning techniques for Android malware detection. Sci Rep. 2024;14(1):10724. doi: 10.1038/s41598-024-60982-y
[76] Zhang J, Zhang C, Liu X, et al. ShadowDroid: practical black-box attack against ML-based Android malware detection. In: 2021 IEEE 27th International Conference on Parallel and Distributed Systems (ICPADS); Beijing, China. IEEE; 2021. p. 629–636.
[77] Šrndi N, Laskov P. Practical evasion of a learning-based classifier: a case study. In: 2014 IEEE symposium on security and privacy, IEEE; San Jose, California, USA; 2014. p. 197–211.
[78] Hu W, Tan Y. Generating adversarial malware examples for black-box attacks based on GAN. In: International Conference on Data Mining and Big Data; Beijing, China. Springer; 2022. p. 409–423.
[79] Wang W, Zhao M, Gao Z, et al. Constructing features for detecting Android malicious applications: issues, taxonomy and directions. IEEE Access. 2019;7:67602–67631. doi: 10.1109/ACCESS.2019.2918139
[80] Doan BG, Nguyen DQ, Montague P, et al. Bayesian learned models can detect adversarial malware for free. 29th European Symposium on Research in Computer Security; Bydgoszcz, Poland. Springer; 2024. Vol. 14982. p. 45–65.
[81] Wang W, Wang X, Feng D, et al. Exploring permission-induced risk in Android applications for malicious application detection. IEEE Trans Inf Forensic Secur. 2014;9(11):1869–1882. doi: 10.1109/TIFS.2014.2353996
[82] Yang J, Soh CK. Structural optimization by genetic algorithms with tournament selection. J Comput Civ Eng. 1997;11(3):195–200. doi: 10.1061/(ASCE)0887-3801(1997)11:3(195)
[83] Allix K, Bissyandé TF, Klein J, et al. Androzoo: collecting millions of Android apps for the research community. In: Proceedings of the 13th international conference on mining software repositories; Austin Texas; 2016. p. 468–471.
[84] Arp D, Spreitzenbarth M, Hübner M, et al. Drebin: effective and explainable detection of android malware in your pocket Network and Distributed System Security (NDSS) Symposium; San Diego, CA, USA; 2014. p. 23–26. doi: 10.14722/ndss.2014.23247
[85] Taha A, Barukab O. Android malware classification using optimized ensemble learning based on genetic algorithms. Sustainability. 2022;14(21):14406. doi: 10.3390/su142114406
[86] Wang L, Gao Y, Gao S, et al. A new feature selection method based on a self-variant genetic algorithm applied to Android malware detection. Symmetry. 2021;13(7):1290. doi: 10.3390/sym13071290
[87] Picek S, Golub M. Comparison of a crossover operator in binary-coded genetic algorithms. WSEAS Trans Comput. 2010;9:1064–1073.
[88] Virusshare. Virusshare. [cited 2024 Oct 11]. Available from: virusshare.com
[89] Basak A. A rank based adaptive mutation in genetic algorithm. Int J Comput Appl. 2020;175(10):975–8887. doi: 10.5120/ijca2020920572
[90] Marsili Libelli S, Alba P. Adaptive mutation in genetic algorithms. Soft Comput. 2000;4(2):76–80. doi: 10.1007/s005000000042
[91] Pendlebury F, Pierazzi F, Jordaney R, et al. Tesseract: eliminating experimental bias in malware classification across space and time. In: 28th USENIX security symposium (USENIX Security 19); Santa Clara, CA; 2019. p. 729–746.
[92] VirusTotal. VirusTotal. [cited 2024 Mar 14]. Available from: https://www.virustotal.com/gui/
[93] Malwarebazaar. Malwarebazaar. [cited 2024 Oct 11]. Available from: https://bazaar.abuse.ch/browse/tag/android/
[94] Ficco M. Malware analysis by combining multiple detectors and observation windows. IEEE Trans Comput. 2021;71:1276–1290. doi: 10.1109/TC.2021.3082002
[95] Rathore H, Sahay SK, Rajvanshi R, et al. Identification of significant permissions for efficient Android malware detection. Lecture notes of the Institute for Computer Sciences, Social-Informatics and Telecommunications Engineering. LNICST. 2021;355:33–52. doi: 10.1007/978-3-030-68737-3_3/COVER
[96] Kim J, Ban Y, Ko E, et al. Mapas: a practical deep learning-based android malware detection system. Int J Inf Secur. 2022;21(4):725–738. doi: 10.1007/s10207-022-00579-6

[97] Meimandi A, Seyfari Y, Lotfi S. Android malware detection using feature selection with hybrid genetic algorithm and simulated annealing. In: Proceedings of the 2020 IEEE 5th Conference on Technology In Electrical and Computer Engineering (ETECH 2020) Information and Communication Technology (ICT); Chongqing, China; 2020.
[98] Sawadogo Z, Dembele JM, Mendy G, et al. Zero-vuln: using deep learning and zero-shot learning techniques to detect zero-day android malware. In: 2023 3rd International Conference on Electrical, Computer, Communications and Mechatronics Engineering (ICECCME) Tenerife, Canary Islands, Spain. IEEE; 2023. p. 1–5.
[99] Korejo I, Yang S, Li C. A comparative study of adaptive mutation operators for genetic algorithms. In: MIC 2009: The VIII Metaheuristics International Conference; Hamburg, Germany; 2009.
[100] Li D, Cui S, Li Y, et al. Pad: towards principled adversarial malware detection against evasion attacks. IEEE Trans Dependable Secure Comput. 2023;21(2):920–936. doi: 10.1109/TDSC.2023.3265665
[101] Chen S, Shen H, Wang R, et al. Towards improving fast adversarial training in multi-exit network. Neural Networks. 2022;150:1–11. doi: 10.1016/j.neunet.2022.02.015
[102] Bostani H, Moonsamy V. Evadedroid: a practical evasion attack on machine learning for black-box Android malware detection. Comput Secur. 2024;139:103676. doi: 10.1016/j.cose.2023.103676
[103] Alshehri A, Hewins A, McCulley M, et al. Risks behind device information permissions in Android OS. Commun Network. 2017;9(4):219–234. doi: 10.4236/cn.2017.94016
[104] Xie N, Wang X, Wang W, et al. Fingerprinting android malware families. Front Comput Sci. 2019;13(3):637–646. doi: 10.1007/s11704-017-6493-y
[105] Zhang H, Qin J, Zhang B, et al. A multiclass detection system for Android malicious apps based on color image features. Wireless Commun Mob Comput. 2020;2020:8882295. doi: 10.1155/2020/8882295
[106] Zhou Y, Jiang X. Dissecting android malware: characterization and evolution. In: 2012 IEEE symposium on security and privacy, IEEE; San Francisco, California; 2012. p. 95–109.
[107] Fortinet Threat Research. Fortinet-reverses-flutter-based-android-malware-fluhorse. [cited 2024 Oct 11]. Available from: https://www.fortinet.com/blog/threat-research/fortinet-reverses-flutter-based-android-malware-fluhorse
[108] Endpointsecurity Threat Research. Android malware targets customers of 30 US banks. [cited 2024 Oct 11]. Available from: https://www.darkreading.com/endpoint-security/xenomorph-android-malware-targets-customers-of-30-us-banks
[109] Cyfirma. Spynote. [cited 2024 Oct 11]. Available from: https://www.cyfirma.com/research/spynote-unmasking-a-sophisticated-android-malware/
[110] Rauti S, Laurén S, Mäki P, et al. Internal interface diversification as a method against malware. J Cyber Secur Technol. 2021;5(1):15–40. doi: 10.1080/23742917.2020.1813397